Dokumentasi Resmi & Sintaks JavaScript (ES2023/ES2024 Up-to

Bahasa: JavaScript | Total Chunk/Dokumen: 83 | Disiapkan untuk RAG System

[javascript_001] JavaScript - JavaScript Introduction

JavaScript Overview & Runtime Introduction: Overview

Sumber: ht

{{PreviousNext("Web/JavaScript/Guide", "Web/JavaScript/Guide/Grammar_and_types")}}
This chapter introduces JavaScript and discusses some of its fundamental concepts.

[javascript_002] JavaScript - JavaScript Introduction

JavaScript Overview & Runtime Introduction: What you should already know

Sumber: ht

What you should already know
This guide assumes you have the following basic background:
- A general understanding of the Internet and the World Wide Web (WWW).
- Good working knowledge of HyperText Markup Language (HTML).
- Some programming experience. If you are new to programming, try one of the tutorials linked on the main page about JavaScript.

[javascript_003] JavaScript - JavaScript Introduction

JavaScript Overview & Runtime Introduction: Where to find JavaScript information

Sumber: ht

Where to find JavaScript information
The JavaScript documentation on MDN includes the following:
- Dynamic scripting with JavaScript provides structured JavaScript guides for beginners and introduces basic concepts of programming and the Internet.
- JavaScript Guide (this guide) provides an overview about the JavaScript language and its objects.
- JavaScript Reference provides detailed reference material for JavaScript.

[javascript_004] JavaScript - JavaScript Introduction

JavaScript Overview & Runtime Introduction: What is JavaScript?

Sumber: ht

What is JavaScript?
JavaScript is a cross-platform, object-oriented scripting language used to make webpages interactive (e.g., having complex animations, clickable buttons, popup menus, etc.). There are also more advanced server side versions of JavaScript such as Node.js, which allow you to add more functionality to a website than downloading files (such as realtime collaboration between multiple computers). Inside a host environment (for example, a web browser), JavaScript can be connected to the objects of its environment to provide programmatic control over them.
JavaScript contains a standard library of objects, such as `Array`, `Map`, and `Math`, and a core set of language elements such as operators, control structures, and statements. Core JavaScript can be extended for a variety of purposes by supplementing it with additional objects; for example:
- _Client-side JavaScript_ extends the core language by supplying objects to control a browser and its _Document Object Model_ (DOM). For example, client-side extensions allow an application to place elements on an HTML form and respond to user events such as mouse clicks, form input, and page navigation.
- _Server-side JavaScript_ extends the core language by supplying objects relevant to running JavaScript on a server. For example, server-side extensions allow an application to communicate with a database, provide continuity of information from one invocation to another of the application, or perform file manipulations on a server.

This means that in the browser, JavaScript can change the way the webpage (DOM) looks. And, likewise, Node.js JavaScript on the server can respond to custom requests sent by code executed in the browser.

[javascript_005] JavaScript - JavaScript Introduction

JavaScript Overview & Runtime Introduction: JavaScript and Java

Sumber: ht

JavaScript and Java

JavaScript and Java are similar in some ways but fundamentally different in some others. The JavaScript language resembles Java but does not have Java's static typing and strong type checking. JavaScript follows most Java expression syntax, naming conventions and basic control-flow constructs which was the reason why it was renamed from LiveScript to JavaScript.

In contrast to Java's compile-time system of classes built by declarations, JavaScript supports a runtime system based on a small number of data types representing numeric, Boolean, and string values. JavaScript has a prototype-based object model instead of the more common class-based object model. The prototype-based model provides dynamic inheritance; that is, what is inherited can vary for individual objects. JavaScript also supports functions without any special declarative requirements. Functions can be properties of objects, executing as loosely typed methods.

JavaScript is a very free-form language compared to Java. You do not have to declare all variables, classes, and methods. You do not have to be concerned with whether methods are public, private, or protected, and you do not have to implement interfaces. Variables, parameters, and function return types are not explicitly typed.

Java is a class-based programming language designed for fast execution and type safety. Type safety means, for instance, that you can't cast a Java integer into an object reference or access private memory by corrupting the Java bytecode. Java's class-based model means that programs consist exclusively of classes and their methods. Java's class inheritance and strong typing generally require tightly coupled object hierarchies. These requirements make Java programming more complex than JavaScript programm

[javascript_006] JavaScript - JavaScript Introduction

JavaScript Overview & Runtime Introduction: JavaScript and the ECMAScript specification

Sumber: ht

JavaScript and the ECMAScript specification

JavaScript is standardized at Ecma International -- the European association for standardizing information and communication systems (ECMA was formerly an acronym for the European Computer Manufacturers Association) to deliver a standardized, international programming language based on JavaScript. This standardized version of JavaScript, called ECMAScript, behaves the same way in all applications that support the standard. Companies can use the open standard language to develop their implementation of JavaScript. The ECMAScript standard is documented in the ECMA-262 specification.

The ECMA-262 standard is also approved by the ISO (International Organization for Standardization) as ISO-16262. You can also find the specification on the Ecma International website. The ECMAScript specification does not describe the Document Object Model (DOM), which is standardized by the World Wide Web Consortium (W3C) and/or WHATWG (Web Hypertext Application Technology Working Group). The DOM defines the way in which HTML document objects are exposed to your script. To get a better idea about the different technologies that are used when programming with JavaScript, consult the article JavaScript technologies overview.

JavaScript documentation versus the ECMAScript specification

The ECMAScript specification is a set of requirements for implementing ECMAScript. It is useful if you want to implement standards-compliant language features in your ECMAScript implementation or engine (such as SpiderMonkey in Firefox, or V8 in Chrome).

The ECMAScript document is _not_ intended to help script programmers. Use the JavaScript documentation for information when writing scripts.

The ECMAScript specification uses terminology and syntax that may be unfa

[javascript_007] JavaScript - JavaScript Introduction

JavaScript Overview & Runtime Introduction: Getting started with JavaScript

Sumber: ht

Getting started with JavaScript
To get started with JavaScript, all you need is a modern web browser. Recent versions of Firefox, Chrome, Microsoft Edge, and Safari all support the features discussed in this guide.
A very useful tool for exploring JavaScript is the JavaScript Console (sometimes called the Web Console, or just the console): this is a tool which enables you to enter JavaScript and run it in the current page.
The screenshots here show the Firefox Web Console, but all modern browsers ship with a console that works in a similar way.
Opening the console
The exact instructions for opening the console vary from one browser to another:
- Opening the console in Firefox
- Opening the console in Chrome
- Opening the console in Microsoft Edge
Entering and running JavaScript
The console appears at the bottom of the browser window. Along the bottom of the console is an input line that you can use to enter JavaScript, and the output appears in the panel above:
!A browser window with the web console open at the bottom, containing two lines of input and output. Text can be entered below that.
The console works the exact same way as `eval`: the last expression entered is returned. For the sake of simplicity, it can be imagined that every time something is entered into the console, it is actually surrounded by `console.log` around `eval`, like so:
Multi-line input in the console
By default, if you press Enter (or Return, depending on your keyboard) after entering a line of code, then the string you typed is executed. To enter multi-line input:
- If the string you typed was incomplete (for example, you typed `function foo() {`) then the console will treat Enter as a line break, and let you type another line.
- If you hold down Shift while pressing Enter, then

Contoh Kode / Perintah CLI:

```
console.log(eval("3 + 5"));

// ----------------------------------------

(function () {
  "use strict";
  /* Start of your code */
  function greetMe(yourName) {
    alert(`Hello ${yourName}`);
  }

  greetMe("World");
  /* End of your code */
})();
```

[javascript_008] JavaScript - JavaScript Introduction

JavaScript Overview & Runtime Introduction: What's next

Sumber: ht

What's next
In the following pages, this guide introduces you to the JavaScript syntax and language features, so that you will be able to write more complex applications.
But for now, remember to always include the `(function(){"use strict";` before your code, and add `})();` to the end of your code. The strict mode and IIFE articles explain what those do, but for now they can be thought of as doing the following:

1. Prevent semantics in JavaScript that trip up beginners.
2. Prevent code snippets executed in the console from interacting with one another (e.g., having something created in one console execution being used for a different console execution).
{{PreviousNext("Web/JavaScript/Guide", "Web/JavaScript/Guide/Grammar_and_types")}}

[javascript_009] JavaScript - Grammar, Variables & Types

Grammar and Types (let, const, Data Types, Type Coercion, Literals): Overview

Sumber: ht

{{PreviousNext("Web/JavaScript/Guide/Introduction", "Web/JavaScript/Guide/Control_flow_and_error_handling")}}
This chapter discusses JavaScript's basic grammar, variable declarations, data types and literals.

[javascript_010] JavaScript - Grammar, Variables & Types

Grammar and Types (let, const, Data Types, Type Coercion, Literals): Basics

Sumber: ht

Basics
JavaScript borrows most of its syntax from Java, C, and C++, but it has also been influenced by Awk, Perl, and Python.
JavaScript is **case-sensitive** and uses the **Unicode** character set. For example, the word Früh (which means "early" in German) could be used as a variable name.
But, the variable `früh` is not the same as `Früh` because JavaScript is case sensitive.
In JavaScript, instructions are called and are separated by semicolons (;).
A semicolon is not necessary after a statement if it is written on its own line. But if more than one statement on a line is desired, then they _must_ be separated by semicolons.
> [!NOTE]
> ECMAScript also has rules for automatic insertion of semicolons (ASI) to end statements. (For more information, see the detailed reference about JavaScript's lexical grammar.)
It is considered best practice, however, to always write a semicolon after a statement, even when it is not strictly needed. This practice reduces the chances of bugs getting into the code.
The source text of JavaScript script gets scanned from left to right, and is converted into a sequence of input elements which are _tokens_, _control characters_, _line terminators_, _comments_, or . (Spaces, tabs, and newline characters are considered whitespace.)

Contoh Kode / Perintah CLI:

```
const Früh = "foobar";
```

[javascript_011] JavaScript - Grammar, Variables & Types

Grammar and Types (let, const, Data Types, Type Coercion, Literals): Comments

Sumber: ht

Comments
The syntax of **comments** is the same as in C++ and in many other languages:
You can't nest block comments. This often happens when you accidentally include a `*/` sequence in your comment, which will terminate the comment.
In this case, you need to break up the `*/` pattern. For example, by inserting a backslash:
Comments behave like whitespace, and are discarded during script execution.
> [!NOTE]
> You might also see a third type of comment syntax at the start of some JavaScript files, which looks something like this: `#!/usr/bin/env node`.
>
> This is called **hashbang comment** syntax, and is a special comment used to specify the path to a particular JavaScript engine that should execute the script. See Hashbang comments for more details.

Contoh Kode / Perintah CLI:

```
// a one line comment
```

```
/* this is a longer,
 * multi-line comment
 */

// ----------------------------------------

In this case, you need to break up the `*/` pattern. For example, by inserting a backslash:
```

[javascript_012] JavaScript - Grammar, Variables & Types

Grammar and Types (let, const, Data Types, Type Coercion, Literals): Declarations

Sumber: ht

Declarations
JavaScript has five kinds of variable declarations.
- {{jsxref("Statements/var", "var")}}
- : Declares a variable, optionally initializing it to a value.
- {{jsxref("Statements/let", "let")}}
- : Declares a block-scoped variable, optionally initializing it to a value.
- {{jsxref("Statements/const", "const")}}
- : Declares a block-scoped variable that cannot be re-assigned, which must be initialized at declaration.
- {{jsxref("Statements/using", "using")}}
- : Declares a variable like `const` that is _synchronously disposed_.
- {{jsxref("Statements/await_using", "await using")}}
- : Declares a variable like `const` that is _asynchronously disposed_.
We will only talk about the first three: `var`, `let`, and `const`, in this article. The `using` and `await using` declarations will be introduced in Resource management.
Variables
You use variables as symbolic names for values in your application. The names of variables, called , conform to certain rules.
A JavaScript identifier usually starts with a letter, underscore (`_`), or dollar sign (`$`). Subsequent characters can also be digits (`0` - `9`). Because JavaScript is case sensitive, letters include the characters `A` through `Z` (uppercase) as well as `a` through `z` (lowercase).
You can use most Unicode letters such as `å` and `ü` in identifiers. (For more details, see the lexical grammar reference.) You can also use Unicode escape sequences to represent characters in identifiers.
Some examples of legal names are `Number_hits`, `temp99`, `$credit`, and `_name`.
Declaring variables
You can declare a variable in two ways:
- With the keyword {{jsxref("Statements/var", "var")}}. For example, `var x = 42`. This syntax can be used to declare both **local** and **global** variables, depending on the _e

Contoh Kode / Perintah CLI:

```
let x;
console.log(x); // logs "undefined"

// ----------------------------------------

### Variable scope

A variable may belong to one of the following [scopes](/en-US/docs/Glossary/Scope):

- Global scope: The default scope for all code running in script mode.
- Module scope: The scope for code running in module mode.
- Function scope: The scope created with a {{Glossary("function")}}.

In addition, variables declared with [`let`](/en-US/docs/Web/JavaScript/Reference/Statements/let) or
```

```
[`const`](/en-US/docs/Web/JavaScript/Reference/Statements/const) can belong to an additional scope:

- Block scope: The scope created with a pair of curly braces (a [block](/en-US/docs/Web/JavaScript/Reference/Statements/block)).

When you declare a variable outside of any function, it is called a _global_ variable, because it is available to any other code in the current document. When you declare a variable within a function, it is called a _local_ variable, because it is available only within that function.

`let` and `const` declarations can also be scoped to the [block statement](/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling#block_statement) that they are declared in.

// ----------------------------------------

However, variables created with `var` are not block-scoped, but only local to the _function (or global scope)_ that the block resides within.

For example, the following code will log `5`, because the scope of `x` is the global context (or the function context if the code is part of a function). The scope of `x` is not limited to the immediate `if` statement block.
```

[javascript_013] JavaScript - Grammar, Variables & Types

Grammar and Types (let, const, Data Types, Type Coercion, Literals): Data structures and types

Sumber: ht

Data structures and types
Data types
The latest ECMAScript standard defines eight data types:
- Seven data types that are :
1. . `true` and `false`.
2. . A special keyword denoting a null value. (Because JavaScript is case-sensitive, `null` is not the same as `Null`, `NULL`, or any other variant.)
3. . A top-level property whose value is not defined.
4. . An integer or floating point number. For example: `42` or `3.14159`.
5. . An integer with arbitrary precision. For example: `9007199254740992n`.
6. . A sequence of characters that represent a text value. For example: `"Howdy"`.
7. Symbol. A data type whose instances are unique and immutable.
- and
Although these data types are relatively few, they enable you to perform useful operations with your applications. Functions are the other fundamental elements of the language. While functions are technically a kind of object, you can think of objects as named containers for values, and functions as procedures that your script can perform.
Data type conversion
JavaScript is a _dynamically typed_ language. This means you don't have to specify the data type of a variable when you declare it. It also means that data types are automatically converted as-needed during script execution.
So, for example, you could define a variable as follows:
And later, you could assign the same variable a string value, for example:
Because JavaScript is dynamically typed, this assignment does not cause an error message.
Numbers and the '+' operator
In expressions involving numeric and string values with the `+` operator, JavaScript converts numeric values to strings. For example, consider the following statements:
With all other operators, JavaScript does _not_ convert numeric values to strings. For example:
Converting strings t

Contoh Kode / Perintah CLI:

```
let answer = 42;

// ----------------------------------------

answer = "Thanks for all the fish!";

// ----------------------------------------

x = "The answer is " + 42; // "The answer is 42"
y = 42 + " is the answer"; // "42 is the answer"
z = "37" + 7; // "377"
```

[javascript_014] JavaScript - Grammar, Variables & Types

Grammar and Types (let, const, Data Types, Type Coercion, Literals): Literals

Sumber: ht

Literals
Literals represent values in JavaScript. These are fixed values -- not variables -- that you _literally_ provide in your script. This section describes the following types of literals:
- Array literals
- Boolean literals
- Numeric literals
- Object literals
- RegExp literals
- String literals
Array literals
An array literal is a list of zero or more expressions, each of which represents an array element, enclosed in square brackets (`[]`). When you create an array using an array literal, it is initialized with the specified values as its elements, and its `length` is set to the number of arguments specified.
The following example creates the `coffees` array with three elements and a `length` of three:
An array literal creates a new array object every time the literal is evaluated. For example, an array defined with a literal in the global scope is created once when the script loads. However, if the array literal is inside a function, a new array is instantiated every time that function is called.
> [!NOTE]
> Array literals create `Array` objects. See and Indexed collections for details on `Array` objects.
Extra commas in array literals
If you put two commas in a row in an array literal, the array leaves an empty slot for the unspecified element. The following example creates the `fish` array:
When you log this array, you will see:
Note that the second item is "empty", which is not exactly the same as the actual `undefined` value. When using array-traversing methods like `Array.prototype.map`, empty slots are skipped. However, index-accessing `fish[1]` still returns `undefined`.
If you include a trailing comma at the end of the list of elements, the comma is ignored.
In the following example, the `length` of the array is three. There is no `myList[3]` a

Contoh Kode / Perintah CLI:

```
const coffees = ["French Roast", "Colombian", "Kona"];

// ----------------------------------------

const fish = ["Lion", , "Angel"];

// ----------------------------------------

console.log(fish);
// [ 'Lion', <1 empty item>, 'Angel' ]
```

[javascript_015] JavaScript - Grammar, Variables & Types

Grammar and Types (let, const, Data Types, Type Coercion, Literals): More information

Sumber: ht

More information
This chapter focuses on basic syntax for declarations and types. To learn more about JavaScript's language constructs, see also the following chapters in this guide:
- Control flow and error handling guide
- Loops and iteration
- Functions
- Expressions and operators guide
In the next chapter, we will have a look at control flow constructs and error handling.
{{PreviousNext("Web/JavaScript/Guide/Introduction", "Web/JavaScript/Guide/Control_flow_and_error_handling")}}

[javascript_016] JavaScript - Control Flow & Error Handling

Control Flow and Error Handling (if, switch, try-catch-finally, Error Types): Overview

Sumber: ht

{{PreviousNext("Web/JavaScript/Guide/Grammar_and_types", "Web/JavaScript/Guide/Loops_and_iteration")}}
JavaScript supports a compact set of statements, specifically
control flow statements, that you can use to incorporate a great deal of interactivity
in your application. This chapter provides an overview of these statements.
The JavaScript reference
contains exhaustive details about the statements in this chapter. The semicolon
(`;`) character is used to separate statements in JavaScript code.
Any JavaScript expression is also a statement.
See Expressions and operators
for complete information about expressions.

[javascript_017] JavaScript - Control Flow & Error Handling

Control Flow and Error Handling (if, switch, try-catch-finally, Error Types): Block statement

Sumber: ht

Block statement
The most basic statement is a _block statement_, which is used to group
statements. The block is delimited by a pair of curly braces:
Example
Block statements are commonly used with control flow statements (`if`,
`for`, `while`).
Here, `{ x++; }` is the block statement.
> [!NOTE]
> `var`-declared variables are not block-scoped, but are scoped to the containing function or script, and the effects of setting them persist beyond the block itself. For example:
>
>
>
> This outputs `2` because the `var x` statement within the block is in the same scope as the `var x` statement before the block. (In C or Java, the equivalent code would have output `1`.)
>
> This scoping effect can be mitigated by using {{jsxref("Statements/let", "let")}} or {{jsxref("Statements/const", "const")}}.

Contoh Kode / Perintah CLI:

```
{
  statement1;
  statement2;
  // ...
  statementN;
}

// ----------------------------------------

while (x < 10) {
  x++;
}

// ----------------------------------------

> var x = 1;
> {
>   var x = 2;
> }
> console.log(x); // 2
>
```

[javascript_018] JavaScript - Control Flow & Error Handling

Control Flow and Error Handling (if, switch, try-catch-finally, Error Types): Conditional statements

Sumber: ht

Conditional statements
A conditional statement is a set of commands that executes if a specified condition is true. JavaScript supports two conditional statements: `if...else` and `switch`.
if...else statement
Use the `if` statement to execute a statement if a logical condition is `true`. Use the optional `else` clause to execute a statement if the condition is `false`.
An `if` statement looks like this:
Here, the `condition` can be any expression that evaluates to `true` or `false`. (See Boolean for an explanation of what evaluates to `true` and `false`.)
If `condition` evaluates to `true`, `statement1` is executed. Otherwise, `statement2` is executed. `statement1` and `statement2` can be any statement, including further nested `if` statements.
You can also compound the statements using `else if` to have multiple conditions tested in sequence, as follows:
In the case of multiple conditions, only the first logical condition which evaluates to `true` will be executed. To execute multiple statements, group them within a block statement (`{ /* ... */ }`).
Best practice
In general, it's good practice to always use block statements -- _especially_ when nesting `if` statements:
In general it's good practice to not have an `if...else` with an assignment like `x = y` as a condition:
However, in the rare case you find yourself wanting to do something like that, the `while` documentation has a Using an assignment as a condition section with guidance on a general best-practice syntax you should know about and follow.

Falsy values

The following values evaluate to `false` (also known as Falsy values):

- `false`
- `undefined`
- `null`
- `0`
- `NaN`
- the empty string (`""`)

All other values -- including all objects -- evaluate to `true` when passed to a conditional statement.

>

Contoh Kode / Perintah CLI:

```
if (condition) {
  statement1;
} else {
  statement2;
}

// ----------------------------------------

if (condition1) {
  statement1;
} else if (condition2) {
  statement2;
} else if (conditionN) {
  statementN;
} else {
  statementLast;
}

// ----------------------------------------

if (condition) {
  // Statements for when condition is true
  // ...
} else {
  // Statements for when condition is false
  // ...
}
```

[javascript_019] JavaScript - Control Flow & Error Handling

Control Flow and Error Handling (if, switch, try-catch-finally, Error Types): Exception handling statements

Sumber: ht

Exception handling statements

You can throw exceptions using the `throw` statement and handle them using the `try...catch` statements.

- `throw` statement
- `try...catch` statement

Exception types

Just about any object can be thrown in JavaScript. Nevertheless, not all thrown objects are created equal. While it is common to throw numbers or strings as errors, it is frequently more effective to use one of the exception types specifically created for this purpose:

- ECMAScript exceptions

- `DOMException`

throw statement

Use the `throw` statement to throw an exception. A `throw`
statement specifies the value to be thrown:

You may throw any expression, not just expressions of a specific type. The following
code throws several exceptions of varying types:

try...catch statement

The `try...catch` statement marks a block of statements to try, and
specifies one or more responses should an exception be thrown. If an exception is
thrown, the `try...catch` statement catches it.

The `try...catch` statement consists of a `try` block, which
contains one or more statements, and a `catch` block, containing statements
that specify what to do if an exception is thrown in the `try` block.

In other words, you want the `try` block to succeed -- but if it does not, you
want control to pass to the `catch` block. If any statement within the
`try` block (or in a function called from within the `try` block)
throws an exception, control _immediately_ shifts to the `catch`
block. If no exception is thrown in the `try` block, the `catch`
block is skipped. The `finally` block executes after the `try` and
`catch` blocks execute but before the statements following the
`try...catch` statement.

The following example uses a `try...catch` statement. The example calls a
function that retrieves a

Contoh Kode / Perintah CLI:

```
throw expression;

// --------------------------------------

throw "Error2"; // String type
throw 42; // Number type
throw true; // Boolean type
throw {
  toString() {
    return "I'm an object!";
  },
};

// --------------------------------------

function getMonthName(mo) {
  mo--; // Adjust month number for array index (so that 0 = Jan, 11 = Dec)
  // prettier-ignore
  const months = [
    "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
  ];
  if (!months[mo]) {
    throw new Error("Invalid month code"); // throw keyword is used here
  }
  return months[mo];
}

try {
  // statements to try
  monthName = getMonthName(myMonth); // function could throw exception
} catch (e) {
  monthName = "unknown";
```

```
  logMyErrors(e); // pass exception object to error handler (i.e. your own function)
}
```

[javascript_020] JavaScript - Functions & Arrow Syntax

Functions (Arrow Functions, Default Parameters, Rest/Spread, Closures): Overview

Sumber: ht

{{PreviousNext("Web/JavaScript/Guide/Loops_and_iteration", "Web/JavaScript/Guide/Expressions_and_operators")}}
Functions are one of the fundamental building blocks in JavaScript. A function in JavaScript is similar to a procedure -- a set of statements that performs a task or calculates a value, but for a procedure to qualify as a function, it should take some input and return an output where there is some obvious relationship between the input and the output. To use a function, you must define it somewhere in the scope from which you wish to call it.
See also the exhaustive reference chapter about JavaScript functions to get to know the details.

[javascript_021] JavaScript - Functions & Arrow Syntax

Functions (Arrow Functions, Default Parameters, Rest/Spread, Closures): Defining functions

Sumber: ht

Defining functions
Function declarations
A **function definition** (also called a **function declaration**, or **function statement**) consists of the `function` keyword, followed by:
- The name of the function.
- A list of parameters to the function, enclosed in parentheses and separated by commas.
- The JavaScript statements that define the function, enclosed in curly braces, `{ /* ... */ }`.
For example, the following code defines a function named `square`:
The function `square` takes one parameter, called `number`. The function consists of one statement that says to return the parameter of the function (that is, `number`) multiplied by itself. The `return` statement specifies the value returned by the function, which is `number * number`.
Parameters are essentially passed to functions **by value** -- so if the code within the body of a function assigns a completely new value to a parameter that was passed to the function, **the change is not reflected globally or in the code which called that function**.
When you pass an object as a parameter, if the function changes the object's properties, that change is visible outside the function, as shown in the following example:
When you pass an array as a parameter, if the function changes any of the array's values, that change is visible outside the function, as shown in the following example:
Function declarations and expressions can be nested, which forms a _scope chain_. For example:
See function scopes and closures for more information.
Function expressions
While the function declaration above is syntactically a statement, functions can also be created by a function expression.
Such a function can be **anonymous**; it does not have to have a name. For example, the function `square` could have been defined as:
H

Contoh Kode / Perintah CLI:

```
function square(number) {
  return number * number;
}

// ---------------------------------------

function myFunc(theObject) {
  theObject.make = "Toyota";
```

```
}

const myCar = {
  make: "Honda",
  model: "Accord",
  year: 1998,
};

console.log(myCar.make); // "Honda"
myFunc(myCar);
console.log(myCar.make); // "Toyota"

// ----------------------------------------

function myFunc(theArr) {
  theArr[0] = 30;
}

const arr = [45];

console.log(arr[0]); // 45
myFunc(arr);
console.log(arr[0]); // 30
```

[javascript_022] JavaScript - Functions & Arrow Syntax

Functions (Arrow Functions, Default Parameters, Rest/Spread, Closures): Calling functions

Sumber: ht

Calling functions

Defining a function does not _execute_ it. Defining it names the function and specifies what to do when the function is called.

Calling the function actually performs the specified actions with the indicated parameters. For example, if you define the function `square`, you could call it as follows:

The preceding statement calls the function with an argument of `5`. The function executes its statements and returns the value `25`.

Functions must be _in scope_ when they are called, but the function declaration can be hoisted (appear below the call in the code). The scope of a function declaration is the function in which it is declared (or the entire program, if it is declared at the top level).

The arguments of a function are not limited to strings and numbers. You can pass whole objects to a function. The `showProps()` function (defined in Working with objects) is an example of a function that takes an object as an argument.

A function can call itself. For example, here is a function that computes factorials recursively:

You could then compute the factorials of `1` through `5` as follows:

There are other ways to call functions. There are often cases where a function needs to be called dynamically, or the number of arguments to a function vary, or in which the context of the function call needs to be set to a specific object determined at runtime.

It turns out that _functions are themselves objects_ -- and in turn, these objects have methods. (See the object.) The `call()` and `apply()` methods can be used to achieve this goal.

Function hoisting

Consider the example below:

This code runs without any error, despite the `square()` function being called before it's declared. This is because the JavaScript interpreter hoists the entire function

Contoh Kode / Perintah CLI:

```
square(5);
```

```
// ----------------------------------------

function factorial(n) {
  if (n === 0 || n === 1) {
    return 1;
  }
  return n * factorial(n - 1);
}

// ----------------------------------------

console.log(factorial(1)); // 1
console.log(factorial(2)); // 2
console.log(factorial(3)); // 6
console.log(factorial(4)); // 24
console.log(factorial(5)); // 120
```

[javascript_023] JavaScript - Functions & Arrow Syntax

Functions (Arrow Functions, Default Parameters, Rest/Spread, Closures): Function scopes and closures

Sumber: ht

Function scopes and closures
Functions form a scope for variables -- this means variables defined inside a function cannot be accessed from anywhere outside the function. The function scope inherits from all the upper scopes. For example, a function defined in the global scope can access all variables defined in the global scope. A function defined inside another function can also access all variables defined in its parent function, and any other variables to which the parent function has access. On the other hand, the parent function (and any other parent scope) does _not_ have access to the variables and functions defined inside the inner function. This provides a sort of encapsulation for the variables in the inner function.
Closures
We also refer to the function body as a _closure_. A closure is any piece of source code (most commonly, a function) that refers to some variables, and the closure "remembers" these variables even when the scope in which these variables were declared has exited.
Closures are usually illustrated with nested functions to show that they remember variables beyond the lifetime of its parent scope; but in fact, nested functions are unnecessary. Technically speaking, all functions in JavaScript form closures -- some just don't capture anything, and closures don't even have to be functions. The key ingredients for a _useful_ closure are the following:
- A parent scope that defines some variables or functions. It should have a clear lifetime, which means it should finish execution at some point. Any scope that's not the global scope satisfies this requirement; this includes blocks, functions, modules, and more.
- An inner scope defined within the parent scope, which refers to some variables or functions defined in the parent scope.
- The inner s

Contoh Kode / Perintah CLI:

```
// The following variables are defined in the global scope
const num1 = 20;
const num2 = 3;
const name = "Chamakh";

// This function is defined in the global scope
function multiply() {
  return num1 * num2;
}

console.log(multiply()); // 60
```

```
// A nested function example
function getScore() {
  const num1 = 2;
  const num2 = 3;

  function add() {
    return `${name} scored ${num1 + num2}`;
  }

  return add();
}

console.log(getScore()); // "Chamakh scored 5"

// ----------------------------------------

// The outer function defines a variable called "name"
const pet = function (name) {
  const getName = function () {
    // The inner function has access to the "name" variable of the outer function
    return name;
  };
  return getName; // Return the inner function, thereby exposing it to outer scopes
};
const myPet = pet("Vivie");

console.log(myPet()); // "Vivie"

// ----------------------------------------

const createPet = function (name) {
  let sex;

  const pet = {
    // setName(newName) is equivalent to setName: function (newName)
    // in this context
    setName(newName) {
      name = newName;
    },

    getName() {
      return name;
    },

    getSex() {
      return sex;
    },

    setSex(newSex) {
      if (
        typeof newSex === "string" &&
        (newSex.toLowerCase() === "male" || newSex.toLowerCase() === "female")
      ) {
        sex = newSex;
      }
    },
  };

  return pet;
};

const pet = createPet("Vivie");
```

```
console.log(pet.getName()); // Vivie

pet.setName("Oliver");
pet.setSex("male");
console.log(pet.getSex()); // male
console.log(pet.getName()); // Oliver
```

[javascript_024] JavaScript - Functions & Arrow Syntax

Functions (Arrow Functions, Default Parameters, Rest/Spread, Closures): Using the arguments object

Sumber: ht

Using the arguments object

The arguments of a function are maintained in an array-like object. Within a function, you can address the arguments passed to it as follows:

where `i` is the ordinal number of the argument, starting at `0`. So, the first argument passed to a function would be `arguments[0]`. The total number of arguments is indicated by `arguments.length`.

Using the `arguments` object, you can call a function with more arguments than it is formally declared to accept. This is often useful if you don't know in advance how many arguments will be passed to the function. You can use `arguments.length` to determine the number of arguments actually passed to the function, and then access each argument using the `arguments` object.

For example, consider a function that concatenates several strings. The only formal argument for the function is a string that specifies the characters that separate the items to concatenate. The function is defined as follows:

You can pass any number of arguments to this function, and it concatenates each argument into a string "list":

> [!NOTE]

> The `arguments` variable is "array-like", but not an array. It is array-like in that it has a numbered index and a `length` property. However, it does _not_ possess all of the array-manipulation methods.

See the object in the JavaScript reference for more information.

Contoh Kode / Perintah CLI:

```
arguments[i];

// ----------------------------------------

function myConcat(separator) {
  let result = ""; // initialize list
  // iterate through arguments
  for (let i = 1; i < arguments.length; i++) {
    result += arguments[i] + separator;
  }
  return result;
}

// ----------------------------------------

console.log(myConcat(", ", "red", "orange", "blue"));
// "red, orange, blue, "

console.log(myConcat("; ", "elephant", "giraffe", "lion", "cheetah"));
// "elephant; giraffe; lion; cheetah; "

console.log(myConcat(". ", "sage", "basil", "oregano", "pepper", "parsley"));
// "sage. basil. oregano. pepper. parsley. "
```

[javascript_025] JavaScript - Functions & Arrow Syntax

Functions (Arrow Functions, Default Parameters, Rest/Spread, Closures): Function parameters

Sumber: ht

Function parameters

There are two special kinds of parameter syntax: _default parameters_ and _rest parameters_.

Default parameters

In JavaScript, parameters of functions default to `undefined`. However, in some situations it might be useful to set a different default value. This is exactly what default parameters do.

In the past, the general strategy for setting defaults was to test parameter values in the body of the function and assign a value if they are `undefined`.

In the following example, if no value is provided for `b`, its value would be `undefined` when evaluating `a*b`, and a call to `multiply` would normally have returned `NaN`. However, this is prevented by the second line in this example:

With _default parameters_, a manual check in the function body is no longer necessary. You can put `1` as the default value for `b` in the function head:

For more details, see default parameters in the reference.

Rest parameters

The rest parameter syntax allows us to represent an indefinite number of arguments as an array.

In the following example, the function `multiply` uses _rest parameters_ to collect arguments from the second one to the end. The function then multiplies these by the first argument.

Contoh Kode / Perintah CLI:

```
function multiply(a, b) {
  b = typeof b !== "undefined" ? b : 1;
  return a * b;
}

console.log(multiply(5)); // 5

// ----------------------------------------

function multiply(a, b = 1) {
  return a * b;
}

console.log(multiply(5)); // 5

// ----------------------------------------

function multiply(multiplier, ...theArgs) {
  return theArgs.map((x) => multiplier * x);
}

const arr = multiply(2, 1, 2, 3);
console.log(arr); // [2, 4, 6]
```

[javascript_026] JavaScript - Functions & Arrow Syntax

Functions (Arrow Functions, Default Parameters, Rest/Spread, Closures): Arrow functions

Sumber: ht

Arrow functions

An arrow function expression (also called a _fat arrow_ to distinguish from a hypothetical `->` syntax in future JavaScript) has a shorter syntax compared to function expressions and does not have its own `this`, `arguments`, `super`, or `new.target`. Arrow functions are always anonymous.

Two factors influenced the introduction of arrow functions: _shorter functions_ and _non-binding_ of `this`.

Shorter functions

In some functional patterns, shorter functions are welcome. Compare:

No separate this

Until arrow functions, every new function defined its own `this` value (a new object in the case of a constructor, undefined in strict mode function calls, the base object if the function is called as an "object method", etc.). This proved to be less than ideal with an object-oriented style of programming.

In ECMAScript 3/5, this issue was fixed by assigning the value in `this` to a variable that could be closed over.

Alternatively, a bound function could be created so that the proper `this` value would be passed to the `growUp()` function.

An arrow function does not have its own `this`; the `this` value of the enclosing execution context is used. Thus, in the following code, the `this` within the function that is passed to `setInterval` has the same value as `this` in the enclosing function:

{{PreviousNext("Web/JavaScript/Guide/Loops_and_iteration", "Web/JavaScript/Guide/Expressions_and_operators")}}

Contoh Kode / Perintah CLI:

```
const a = ["Hydrogen", "Helium", "Lithium", "Beryllium"];

const a2 = a.map(function (s) {
  return s.length;
});

console.log(a2); // [8, 6, 7, 9]

const a3 = a.map((s) => s.length);

console.log(a3); // [8, 6, 7, 9]

// ----------------------------------------

function Person() {
  // The Person() constructor defines `this` as itself.
  this.age = 0;

  setInterval(function growUp() {
    // In nonstrict mode, the growUp() function defines `this`
    // as the global object, which is different from the `this`
    // defined by the Person() constructor.
    this.age++;
  }, 1000);
}

const p = new Person();

// ----------------------------------------

function Person() {
  // Some choose `that` instead of `self`.
  // Choose one and be consistent.
  const self = this;
  self.age = 0;

  setInterval(function growUp() {
    // The callback refers to the `self` variable of which
    // the value is the expected object.
    self.age++;
  }, 1000);
}
```

[javascript_027] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Overview

Sumber: ht

{{PreviousNext("Web/JavaScript/Guide/Working_with_objects", "Web/JavaScript/Guide/Using_promises")}}
JavaScript is a prototype-based language -- an object's behaviors are specified by its own properties and its prototype's properties. However, with the addition of classes, the creation of hierarchies of objects and the inheritance of properties and their values are much more in line with other object-oriented languages such as Java. In this section, we will demonstrate how objects can be created from classes.
In many other languages, _classes_, or constructors, are clearly distinguished from _objects_, or instances. In JavaScript, classes are mainly an abstraction over the existing prototypical inheritance mechanism -- all patterns are convertible to prototype-based inheritance. Classes themselves are normal JavaScript values as well, and have their own prototype chains. In fact, most plain JavaScript functions can be used as constructors -- you use the `new` operator with a constructor function to create a new object.
We will be playing with the well-abstracted class model in this tutorial, and discuss what semantics classes offer. If you want to dive deep into the underlying prototype system, you can read the Inheritance and the prototype chain guide.
This chapter assumes that you are already somewhat familiar with JavaScript and that you have used ordinary objects.

[javascript_028] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Overview of classes

Sumber: ht

Overview of classes
If you have some hands-on experience with JavaScript, or have followed along with the guide, you probably have already used classes, even if you haven't created one. For example, this may seem familiar to you:
On the first line, we created an instance of the class `Date`, and called it `bigDay`. On the second line, we called a method `toLocaleDateString()` on the `bigDay` instance, which returns a string. Then, we compared two numbers: one returned from the `getTime()` method, the other directly called from the `Date` class _itself_, as `Date.now()`.
`Date` is a built-in class of JavaScript. From this example, we can get some basic ideas of what classes do:
- Classes create objects through the `new` operator.
- Each object has some properties (data or method) added by the class.
- The class stores some properties (data or method) itself, which are usually used to interact with instances.
These correspond to the three key features of classes:
- Constructor;
- Instance methods and instance fields;
- Static methods and static fields.

Contoh Kode / Perintah CLI:

```
const bigDay = new Date(2019, 6, 19);
console.log(bigDay.toLocaleDateString());
if (bigDay.getTime() < Date.now()) {
  console.log("Once upon a time...");
}
```

[javascript_029] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Declaring a class

Sumber: ht

Declaring a class
Classes are usually created with _class declarations_.
Within a class body, there are a range of features available.
If you came from a pre-ES6 world, you may be more familiar with using functions as constructors. The pattern above

would roughly translate to the following with function constructors:

> [!NOTE]
> Private fields and methods are new features in classes with no trivial equivalent in function constructors.

Constructing a class

After a class has been declared, you can create instances of it using the `new` operator.

Typical function constructors can both be constructed with `new` and called without `new`. However, attempting to "call" a class without `new` will result in an error.

Class declaration hoisting

Unlike function declarations, class declarations are not hoisted (or, in some interpretations, hoisted but with the temporal dead zone restriction), which means you cannot use a class before it is declared.

This behavior is similar to variables declared with `let` and `const`.

Class expressions

Similar to functions, class declarations also have their expression counterparts.

Class expressions can have names as well. The expression's name is only visible to the class's body.

Contoh Kode / Perintah CLI:

```
class MyClass {
  // class body...
}

// ----------------------------------------

class MyClass {
  // Constructor
  constructor() {
    // Constructor body
  }
  // Instance field
  myField = "foo";
  // Instance method
  myMethod() {
    // myMethod body
  }
  // Static field
  static myStaticField = "bar";
  // Static method
  static myStaticMethod() {
    // myStaticMethod body
  }
  // Static block
  static {
    // Static initialization code
  }
  // Fields, methods, static fields, and static methods all have
  // "private" forms
  #myPrivateField = "bar";
}

// ----------------------------------------

function MyClass() {
  this.myField = "foo";
  // Constructor body
}
MyClass.myStaticField = "bar";
MyClass.myStaticMethod = function () {
  // myStaticMethod body
};
MyClass.prototype.myMethod = function () {
  // myMethod body
```

```
};

(function () {
  // Static initialization code
})();
```

[javascript_030] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Constructor

Sumber: ht

Constructor

Perhaps the most important job of a class is to act as a "factory" for objects. For example, when we use the `Date` constructor, we expect it to give a new object which represents the date data we passed in -- which we can then manipulate with other methods the instance exposes. In classes, the instance creation is done by the constructor.

As an example, we would create a class called `Color`, which represents a specific color. Users create colors through passing in an RGB triplet.

Open your browser's devtools, paste the above code into the console, and then create an instance:

You should see some output like this:

You have successfully created a `Color` instance, and the instance has a `values` property, which is an array of the RGB values you passed in. That is pretty much equivalent to the following:

The constructor's syntax is exactly the same as a normal function -- which means you can use other syntaxes, like rest parameters:

Each time you call `new`, a different instance is created.

Within a class constructor, the value of `this` points to the newly created instance. You can assign properties to it, or read existing properties (especially methods -- which we will cover next).

The `this` value will be automatically returned as the result of `new`. You are advised to not return any value from the constructor -- because if you return a non-primitive value, it will become the value of the `new` expression, and the value of `this` is dropped. (You can read more about what `new` does in its description.)

Contoh Kode / Perintah CLI:

```
class Color {
  constructor(r, g, b) {
    // Assign the RGB values as a property of `this`.
    this.values = [r, g, b];
  }
}

// ----------------------------------------

const red = new Color(255, 0, 0);
console.log(red);

// ----------------------------------------

You have successfully created a `Color` instance, and the instance has a `values` property, which is an array of the RGB values you passed in. That is pretty much equivalent to the following:
```

[javascript_031] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Instance methods

Sumber: ht

Instance methods

If a class only has a constructor, it is not much different from a `createX` factory function which just creates plain objects. However, the power of classes is that they can be used as "templates" which automatically assign methods to instances.

For example, for `Date` instances, you can use a range of methods to get different information from a single date value, such as the year, month, day of the week, etc. You can also set those values through the `setX` counterparts like `setFullYear`.

For our own `Color` class, we can add a method called `getRed` which returns the red value of the color.

Without methods, you may be tempted to define the function within the constructor:

This also works. However, a problem is that this creates a new function every time a `Color` instance is created, even when they all do the same thing!

In contrast, if you use a method, it will be shared between all instances. A function can be shared between all instances, but still have its behavior differ when different instances call it, because the value of `this` is different. If you are curious _where_ this method is stored in -- it's defined on the prototype of all instances, or `Color.prototype`, which is explained in more detail in Inheritance and the prototype chain.

Similarly, we can create a new method called `setRed`, which sets the red value of the color.

Contoh Kode / Perintah CLI:

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  getRed() {
    return this.values[0];
  }
}

const red = new Color(255, 0, 0);
console.log(red.getRed()); // 255

// ----------------------------------------

class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
    this.getRed = function () {
      return this.values[0];
    };
  }
}

// ----------------------------------------

console.log(new Color().getRed === new Color().getRed); // false
```

[javascript_032] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Private fields

Sumber: ht

Private fields

You might be wondering: why do we want to go to the trouble of using `getRed` and `setRed` methods, when we can directly access the `values` array on the instance?

There is a philosophy in object-oriented programming called "encapsulation". This means you should not access the underlying implementation of an object, but instead use well-abstracted methods to interact with it. For example, if we suddenly decided to represent colors as HSL instead:

The user assumption that `values` means the RGB value suddenly collapses, and it may cause their logic to break. So, if you are an implementor of a class, you would want to hide the internal data structure of your instance from your user, both to keep the API clean and to prevent the user's code from breaking when you do some "harmless refactors". In classes, this is done through _private fields_.

A private field is an identifier prefixed with `#` (the hash symbol). The hash is an integral part of the field's name, which means a private field can never clash names with a public field or method. In order to refer to a private field anywhere in the class, you must _declare_ it in the class body (you can't create a private element on the fly). Apart from this, a private field is pretty much equivalent to a normal property.

Accessing private fields outside the class is an early syntax error. The language can guard against this because `#privateField` is a special syntax, so it can do some static analysis and find all usage of private fields before even evaluating the code.

> [!NOTE]

> Code run in the Chrome console can access private elements outside the class. This is a DevTools-only relaxation of the JavaScript syntax restriction.

Private fields in JavaScript are _hard private_: if the class does not implement m

Contoh Kode / Perintah CLI:

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
}

const red = new Color(255, 0, 0);
red.values[0] = 0;
console.log(red.values[0]); // 0

// ----------------------------------------

class Color {
  constructor(r, g, b) {
    // values is now an HSL array!
    this.values = rgbToHSL([r, g, b]);
  }
  getRed() {
    return hslToRGB(this.values)[0];
  }
  setRed(value) {
    const rgb = hslToRGB(this.values);
    rgb[0] = value;
    this.values = rgbToHSL(rgb);
  }
}

const red = new Color(255, 0, 0);
console.log(red.values[0]); // 0; It's not 255 anymore, because the H value for pure red is 0
```

```
// ----------------------------------------

class Color {
  // Declare: every Color instance has a private field called #values.
  #values;
  constructor(r, g, b) {
    this.#values = [r, g, b];
  }
  getRed() {
    return this.#values[0];
  }
  setRed(value) {
    this.#values[0] = value;
  }
}

const red = new Color(255, 0, 0);
console.log(red.getRed()); // 255
```

[javascript_033] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Accessor fields

Sumber: ht

Accessor fields

`color.getRed()` and `color.setRed()` allow us to read and write to the red value of a color. If you come from languages like Java, you will be very familiar with this pattern. However, using methods to simply access a property is still somewhat unergonomic in JavaScript. _Accessor fields_ allow us to manipulate something as if it is an "actual property".

It looks as if the object has a property called `red` -- but actually, no such property exists on the instance! There are only two methods, but they are prefixed with `get` and `set`, which allows them to be manipulated as if they were properties.

If a field only has a getter but no setter, it will be effectively read-only.

In strict mode, the `red.red = 0` line will throw a type error: "Cannot set property red of #\ which has only a getter". In non-strict mode, the assignment is silently ignored.

Contoh Kode / Perintah CLI:

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  get red() {
    return this.values[0];
  }
  set red(value) {
    this.values[0] = value;
  }
}

const red = new Color(255, 0, 0);
red.red = 0;
console.log(red.red); // 0

// ----------------------------------------

class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
```

```
  get red() {
    return this.values[0];
  }
}

const red = new Color(255, 0, 0);
red.red = 0;
console.log(red.red); // 255
```

[javascript_034] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Public fields

Sumber: ht

Public fields

Private fields also have their public counterparts, which allow every instance to have a property. Fields are usually designed to be independent of the constructor's parameters.

Public fields are almost equivalent to assigning a property to `this`. For example, the above example can also be converted to:

Contoh Kode / Perintah CLI:

```
class MyClass {
  luckyNumber = Math.random();
}
console.log(new MyClass().luckyNumber); // 0.5
console.log(new MyClass().luckyNumber); // 0.3

// ----------------------------------------

class MyClass {
  constructor() {
    this.luckyNumber = Math.random();
  }
}
```

[javascript_035] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Static properties

Sumber: ht

Static properties

With the `Date` example, we have also encountered the `Date.now()` method, which returns the current date. This method does not belong to any date instance -- it belongs to the class itself. However, it's put on the `Date` class instead of being exposed as a global `DateNow()` function, because it's mostly useful when dealing with date instances.

> [!NOTE]

> Prefixing utility methods with what they deal with is called "namespacing" and is considered a good practice. For example, in addition to the older, unprefixed `parseInt()` method, JavaScript also later added the prefixed `Number.parseInt()` method to indicate that it's for dealing with numbers.

Static properties are a group of class features that are defined on the class itself, rather than on individual instances of the class. These features include:

- Static methods
- Static fields
- Static getters and setters

Everything also has private counterparts. For example, for our `Color` class, we can create a static method that checks whether a given triplet is a valid RGB value:

Static properties are very similar to their instance counterparts, except that:

- They are all prefixed with `static`, and
- They are not accessible from instances.

There is also a special construct called a _static initialization block_, which is a block of code that runs when the class is first loaded.

Static initialization blocks are almost equivalent to immediately executing some code after a class has been declared. The only difference is that they have access to static private elements.

Contoh Kode / Perintah CLI:

```
class Color {
  static isValid(r, g, b) {
    return r >= 0 && r <= 255 && g >= 0 && g <= 255 && b >= 0 && b <= 255;
  }
}

Color.isValid(255, 0, 0); // true
Color.isValid(1000, 0, 0); // false

// ----------------------------------------

console.log(new Color(0, 0, 0).isValid); // undefined

// ----------------------------------------

class MyClass {
  static {
    MyClass.myStaticProperty = "foo";
  }
}

console.log(MyClass.myStaticProperty); // 'foo'
```

[javascript_036] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Extends and inheritance

Sumber: ht

Extends and inheritance

A key feature that classes bring about (in addition to ergonomic encapsulation with private fields) is _inheritance_, which means one object can "borrow" a large part of another object's behaviors, while overriding or enhancing certain parts with its own logic.

For example, suppose our `Color` class now needs to support transparency. We may be tempted to add a new field that indicates its transparency:

However, this means every instance -- even the vast majority which aren't transparent (those with an alpha value of 1) -- will have to have the extra alpha value, which is not very elegant. Plus, if the features keep growing, our `Color` class will become very bloated and hard to maintain.

Instead, in object-oriented programming, we would create a _derived class_. The derived class has access to all public properties of the parent class. In JavaScript, derived classes are declared with an `extends` clause, which indicates the class it extends from.

There are a few things that have immediately come to attention. First is that in the constructor, we are calling `super(r, g, b)`. It is a language requirement to call `super()` before accessing `this`. The `super()` call calls the parent class's constructor to initialize `this` -- here it's roughly equivalent to `this = new Color(r, g, b)`. You can have code before `super()`, but you cannot access `this` before `super()` -- the language prevents you from accessing the uninitialized `this`.

After the parent class is done with modifying `this`, the derived class can do its own logic. Here we added a private field called `#alpha`, and also provided a pair of getter/setters to interact with them.

A derived class inherits all methods from its parent. For example, consider the `get red()` accessor we added to

Contoh Kode / Perintah CLI:

```
class Color {
  #values;
  constructor(r, g, b, a = 1) {
    this.#values = [r, g, b, a];
  }
  get alpha() {
    return this.#values[3];
  }
  set alpha(value) {
    if (value < 0 || value > 1) {
      throw new RangeError("Alpha value must be between 0 and 1");
    }
    this.#values[3] = value;
  }
}

// ----------------------------------------

class ColorWithAlpha extends Color {
  #alpha;
  constructor(r, g, b, a) {
    super(r, g, b);
    this.#alpha = a;
  }
  get alpha() {
    return this.#alpha;
  }
  set alpha(value) {
    if (value < 0 || value > 1) {
      throw new RangeError("Alpha value must be between 0 and 1");
    }
    this.#alpha = value;
  }
}

// ----------------------------------------

const color = new ColorWithAlpha(255, 0, 0, 0.5);
console.log(color.red); // 255
```

[javascript_037] JavaScript - Classes & Modern OOP

Using Classes (Constructors, Private Fields #, Static Properties, Inheritance): Why classes?

Sumber: ht

Why classes?

The guide has been pragmatic so far: we are focusing on _how_ classes can be used, but there's one question unanswered: _why_ would one use a class? The answer is: it depends.

Classes introduce a _paradigm_, or a way to organize your code. Classes are the foundations of object-oriented programming, which is built on concepts like [inheritance]() and [polymorphism]() (especially _subtype polymorphism_). However, many people are philosophically against certain OOP practices and don't use classes as a result.

For example, one thing that makes `Date` objects infamous is that they're _mutable_.

Mutability and internal state are important aspects of object-oriented programming, but often make code hard to reason with -- because any seemingly innocent operation may have unexpected side effects and change the behavior in other parts of the program.

In order to reuse code, we usually resort to extending classes, which can create big hierarchies of inheritance patterns.

!A typical OOP inheritance tree, with five classes and three levels

However, it is often hard to describe inheritance cleanly when one class can only extend one other class. Often, we want the behavior of multiple classes. In Java, this is done through interfaces; in JavaScript, it can be done through mixins. But at the end of the day, it's still not very convenient.

On the brighter side, classes are a very powerful way to organize our code on a higher level. For example, without the `Color` class, we may need to create a dozen of utility functions:

But with classes, we can congregate them all under the `Color` namespace, which improves readability. In addition, the introduction of private fields allows us to hide certain data from downstream users, creating a clean API.

In general, you should consi

Contoh Kode / Perintah CLI:

```
function incrementDay(date) {
  return new Date(date.setDate(date.getDate() + 1));
}
const date = new Date(); // 2019-06-19
const newDay = incrementDay(date);
console.log(newDay); // 2019-06-20
// The old date is modified as well!?
console.log(date); // 2019-06-20

// ----------------------------------------

function isRed(color) {
  return color.red === 255;
}
function isValidColor(color) {
  return (
    color.red >= 0 &&
    color.red <= 255 &&
    color.green >= 0 &&
    color.green <= 255 &&
    color.blue >= 0 &&
    color.blue <= 255
  );
}
// ...
```

[javascript_038] JavaScript - Asynchronous JavaScript (Promises & Async/Await)

Using Promises and Async/Await (Chaining, Promise.all, Error Handling): Overview

Sumber: ht

{{PreviousNext("Web/JavaScript/Guide/Using_classes", "Web/JavaScript/Guide/Typed_arrays")}}

A is an object representing the eventual completion or failure of an asynchronous operation. Since most people are consumers of already-created promises, this guide will explain consumption of returned promises before explaining how to create them.

Essentially, a promise is a returned object to which you attach callbacks, instead of passing callbacks into a function. Imagine a function, `createAudioFileAsync()`, which asynchronously generates a sound file given a configuration record and two callback functions: one called if the audio file is successfully created, and the other called if an error occurs.

Here's some code that uses `createAudioFileAsync()`:

If `createAudioFileAsync()` were rewritten to return a promise, you would attach your callbacks to it instead:

This convention has several advantages. We will explore each one.

Contoh Kode / Perintah CLI:

```
function successCallback(result) {
  console.log(`Audio file ready at URL: ${result}`);
}

function failureCallback(error) {
  console.error(`Error generating audio file: ${error}`);
}
```

```
createAudioFileAsync(audioSettings, successCallback, failureCallback);

// --------------------------------------

createAudioFileAsync(audioSettings).then(successCallback, failureCallback);
```

[javascript_039] JavaScript - Asynchronous JavaScript (Promises & Async/Await)

Using Promises and Async/Await (Chaining, Promise.all, Error Handling): Chaining

Sumber: ht

Chaining

A common need is to execute two or more asynchronous operations back to back, where each subsequent operation starts when the previous operation succeeds, with the result from the previous step. In the old days, doing several asynchronous operations in a row would lead to the classic callback hell:

With promises, we accomplish this by creating a promise chain. The API design of promises makes this great, because callbacks are attached to the returned promise object, instead of being passed into a function.

Here's the magic: the `then()` function returns a **new promise**, different from the original:

This second promise (`promise2`) represents the completion not just of `doSomething()`, but also of the `successCallback` or `failureCallback` you passed in -- which can be other asynchronous functions returning a promise. When that's the case, any callbacks added to `promise2` get queued behind the promise returned by either `successCallback` or `failureCallback`.

> [!NOTE]
> If you want a working example to play with, you can use the following template to create any function returning a promise:
>
>
>
> The implementation is discussed in the Creating a Promise around an old callback API section below.

With this pattern, you can create longer chains of processing, where each promise represents the completion of one asynchronous step in the chain. In addition, the arguments to `then` are optional, and `catch(failureCallback)` is short for `then(null, failureCallback)` -- so if your error handling code is the same for all steps, you can attach it to the end of the chain:

You might see this expressed with arrow functions instead:

> [!NOTE]
> Arrow function expressions can have an implicit return; so, `() => x` is short for `() => { return x; }`.

`doSomethingElse` an

Contoh Kode / Perintah CLI:

```
With promises, we accomplish this by creating a promise chain. The API design of promises makes this great, because callbacks are attached to the returned promise object, instead of being passed into a function.

Here's the magic: the `then()` function returns a **new promise**, different from the original:

// --------------------------------------

This second promise (`promise2`) represents the completion not just of `doSomething()`, but also of the `successCallback` or `failureCallback` you passed in -- which can be other asynchronous functions returning a promise. When that's the case, any callbacks added to `promise2` get queued behind the promise returned by either `successCallback` or `failureCallback`.

> [!NOTE]
> If you want a working example to play with, you can use the following template to create any function returning a promise:
>
>
```

```
// ----------------------------------------

>
>  The  implementation  is  discussed  in  the  [Creating  a  Promise  around  an  old  callback
API](#creating_a_promise_around_an_old_callback_api) section below.

With  this  pattern,  you  can  create  longer  chains  of  processing,  where  each  promise  represents  the
completion of one asynchronous step in the chain. In addition, the arguments to `then` are optional, and
`catch(failureCallback)` is short for `then(null, failureCallback)`  --  so if your error handling code
is the same for all steps, you can attach it to the end of the chain:
```

[javascript_040] JavaScript - Asynchronous JavaScript (Promises & Async/Await)

Using Promises and Async/Await (Chaining, Promise.all, Error Handling): Error handling

Sumber: ht

Error handling

You might recall seeing `failureCallback` three times in the pyramid of doom earlier, compared to only once at the end of the promise chain:

If there's an exception, the browser will look down the chain for `.catch()` handlers or `onRejected`. This is very much modeled after how synchronous code works:

This symmetry with asynchronous code culminates in the `async`/`await` syntax:

Promises solve a fundamental flaw with the callback pyramid of doom, by catching all errors, even thrown exceptions and programming errors. This is essential for functional composition of asynchronous operations. All errors are now handled by the `catch()` method at the end of the chain, and you should almost never need to use `try`/`catch` without using `async`/`await`.

Nesting

In the examples above involving `listOfIngredients`, the first one has one promise chain nested in the return value of another `then()` handler, while the second one uses an entirely flat chain. Simple promise chains are best kept flat without nesting, as nesting can be a result of careless composition.

Nesting is a control structure to limit the scope of `catch` statements. Specifically, a nested `catch` only catches failures in its scope and below, not errors higher up in the chain outside the nested scope. When used correctly, this gives greater precision in error recovery:

Note that the optional steps here are nested -- with the nesting caused not by the indentation, but by the placement of the outer `(` and `)` parentheses around the steps.

The inner error-silencing `catch` handler only catches failures from `doSomethingOptional()` and `doSomethingExtraNice()`, after which the code resumes with `moreCriticalStuff()`. Importantly, if `doSomethingCritical()` fails, its error is caught by the fina

Contoh Kode / Perintah CLI:

```
doSomething()
  .then((result) => doSomethingElse(result))
  .then((newResult) => doThirdThing(newResult))
  .then((finalResult) => console.log(`Got the final result: ${finalResult}`))
  .catch(failureCallback);

// ----------------------------------------

try {
  const result = syncDoSomething();
  const newResult = syncDoSomethingElse(result);
  const finalResult = syncDoThirdThing(newResult);
  console.log(`Got the final result: ${finalResult}`);
} catch (error) {
  failureCallback(error);
}
```

```
// ----------------------------------------

async function foo() {
  try {
    const result = await doSomething();
    const newResult = await doSomethingElse(result);
    const finalResult = await doThirdThing(newResult);
    console.log(`Got the final result: ${finalResult}`);
  } catch (error) {
    failureCallback(error);
  }
}
```

[javascript_041] JavaScript - Asynchronous JavaScript (Promises & Async/Await)

Using Promises and Async/Await (Chaining, Promise.all, Error Handling): Composition

Sumber: ht

Composition

There are four composition tools for running asynchronous operations concurrently: , , , and .

We can start operations at the same time and wait for them all to finish like this:

If one of the promises in the array rejects, `Promise.all()` immediately rejects the returned promise. The other operations continue to run, but their outcomes are not available via the return value of `Promise.all()`. This may cause unexpected state or behavior. is another composition tool that ensures all operations are complete before resolving.

These methods all run promises concurrently -- a sequence of promises are started simultaneously and do not wait for each other. Sequential composition is possible using some clever JavaScript:

In this example, we reduce an array of asynchronous functions down to a promise chain. The code above is equivalent to:

This can be made into a reusable compose function, which is common in functional programming:

The `composeAsync()` function accepts any number of functions as arguments and returns a new function that accepts an initial value to be passed through the composition pipeline:

Sequential composition can also be done more succinctly with async/await:

However, before you compose promises sequentially, consider if it's really necessary -- it's always better to run promises concurrently so that they don't unnecessarily block each other unless one promise's execution depends on another's result.

Contoh Kode / Perintah CLI:

```
Promise.all([func1(), func2(), func3()]).then(([result1, result2, result3]) => {
  // use result1, result2 and result3
});

// ----------------------------------------

[func1, func2, func3]
  .reduce((p, f) => p.then(f), Promise.resolve())
  .then((result3) => {
    /* use result3 */
  });

// ----------------------------------------

Promise.resolve()
  .then(func1)
  .then(func2)
  .then(func3)
  .then((result3) => {
    /* use result3 */
  });
```

[javascript_042] JavaScript - Asynchronous JavaScript (Promises & Async/Await)

Using Promises and Async/Await (Chaining, Promise.all, Error Handling): Cancellation

Sumber: ht

Cancellation
`Promise` itself has no first-class protocol for cancellation, but you may be able to directly cancel the underlying asynchronous operation, typically using `AbortController`.

[javascript_043] JavaScript - Asynchronous JavaScript (Promises & Async/Await)

Using Promises and Async/Await (Chaining, Promise.all, Error Handling): Creating a Promise around an old callback API

Sumber: ht

Creating a Promise around an old callback API
A can be created from scratch using its constructor. This should be needed only to wrap old APIs.
In an ideal world, all asynchronous functions would already return promises. Unfortunately, some APIs still expect success and/or failure callbacks to be passed in the old way. The most obvious example is the function:
Mixing old-style callbacks and promises is problematic. If `saySomething()` fails or contains a programming error, nothing catches it. This is intrinsic to the design of `setTimeout()`.
Luckily we can wrap `setTimeout()` in a promise. The best practice is to wrap the callback-accepting functions at the lowest possible level, and then never call them directly again:
The promise constructor takes an executor function that lets us resolve or reject a promise manually. Since `setTimeout()` doesn't really fail, we left out reject in this case. For more information on how the executor function works, see the `Promise()` reference.

Contoh Kode / Perintah CLI:

```
setTimeout(() => saySomething("10 seconds passed"), 10 * 1000);

// ----------------------------------------

const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

wait(10 * 1000)
  .then(() => saySomething("10 seconds"))
  .catch(failureCallback);
```

[javascript_044] JavaScript - Asynchronous JavaScript (Promises & Async/Await)

Using Promises and Async/Await (Chaining, Promise.all, Error Handling): Timing

Sumber: ht

Timing
Lastly, we will look into the more technical details, about when the registered callbacks get called.
Guarantees
In the callback-based API, when and how the callback gets called depends on the API implementor. For example, the callback may be called synchronously or asynchronously:
The above design is strongly discouraged because it leads to the so-called "state of Zalgo". In the context of designing asynchronous APIs, this means a callback is called synchronously in some cases but asynchronously in other cases, creating ambiguity for the caller. For further background, see the article Designing APIs for Asynchrony, where the term was first formally presented. This API design makes side effects hard to analyze:
On the other hand, promises are a form of inversion of control -- the API implementor does not control when the callback gets called. Instead, the job of maintaining the callback queue and deciding when to call the callbacks is delegated to the promise implementation, and both the API user and API developer automatically get strong semantic guarantees, including:
- Callbacks added with `then()` will never be invoked before the completion of the current run of the JavaScript event loop.

- These callbacks will be invoked even if they were added _after_ the success or failure of the asynchronous operation that the promise represents.
- Multiple callbacks may be added by calling `then()` several times. They will be invoked one after another, in the order in which they were inserted.
To avoid surprises, functions passed to `then()` will never be called synchronously, even with an already-resolved promise:
Instead of running immediately, the passed-in function is put on a microtask queue, which means it runs later (only after the function which created it

Contoh Kode / Perintah CLI:

```
The above design is strongly discouraged because it leads to the so-called "state of Zalgo". In the context of designing asynchronous APIs, this means a callback is called synchronously in some cases but asynchronously in other cases, creating ambiguity for the caller. For further background, see the article [Designing APIs for Asynchrony](https://blog.izs.me/2013/08/designing-apis-for-asynchrony/), where the term was first formally presented. This API design makes side effects hard to analyze:

// ----------------------------------------

On the other hand, promises are a form of [inversion of control](https://en.wikipedia.org/wiki/Inversion_of_control) -- the API implementor does not control when the callback gets called. Instead, the job of maintaining the callback queue and deciding when to call the callbacks is delegated to the promise implementation, and both the API user and API developer automatically get strong semantic guarantees, including:

- Callbacks added with [`then()`](/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/then) will never be invoked before the [completion of the current run](/en-US/docs/Web/JavaScript/Reference/Execution_model#run-to-completion) of the JavaScript event loop.
- These callbacks will be invoked even if they were added _after_ the success or failure of the asynchronous operation that the promise represents.
- Multiple callbacks may be added by calling [`then()`](/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/then) several times. They will be invoked one after another, in the order in which they were inserted.

To avoid surprises, functions passed to [`then()`](/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/then) will never be called synchronously, even with an already-resolved promise:

// ----------------------------------------

Instead of running immediately, the passed-in function is put on a microtask queue, which means it runs later (only after the function which created it exits, and when the JavaScript execution stack is empty), just before control is returned to the event loop; i.e., pretty soon:
```

[javascript_045] JavaScript - Iterators and Generators

Iterators and Generators (yield, Iterables, for...of loops): Overview

Sumber: ht

{{PreviousNext("Web/JavaScript/Guide/Typed_arrays", "Web/JavaScript/Guide/Resource_management")}}
Iterators and Generators bring the concept of iteration directly into the core language and provide a mechanism for customizing the behavior of {{jsxref("Statements/for...of", "for...of")}} loops.
For details, see also:
- Iteration protocols
- {{jsxref("Statements/for...of", "for...of")}}
- {{jsxref("Statements/function*", "function*")}} and
- {{jsxref("Operators/yield", "yield")}} and {{jsxref("Operators/yield*", "yield*")}}

[javascript_046] JavaScript - Iterators and Generators

Iterators and Generators (yield, Iterables, for...of loops): Iterators

Sumber: ht

Iterators

In JavaScript an **iterator** is an object which defines a sequence and potentially a return value upon its termination. Specifically, an iterator is any object which implements the Iterator protocol by having a `next()` method that returns an object with two properties:

- `value`
- : The next value in the iteration sequence.
- `done`
- : This is `true` if the last value in the sequence has already been consumed. If `value` is present alongside `done`, it is the iterator's return value.

Once created, an iterator object can be iterated explicitly by repeatedly calling `next()`. Iterating over an iterator is said to consume the iterator, because it is generally only possible to do once. After a terminating value has been yielded additional calls to `next()` should continue to return `{done: true}`.

The most common iterator in JavaScript is the Array iterator, which returns each value in the associated array in sequence.

While it is easy to imagine that all iterators could be expressed as arrays, this is not true. Arrays must be allocated in their entirety, but iterators are consumed only as necessary. Because of this, iterators can express sequences of unlimited size, such as the range of integers between `0` and .

Here is an example which can do just that. It allows creation of a range iterator which defines a sequence of integers from `start` (inclusive) to `end` (exclusive) spaced `step` apart. Its final return value is the size of the sequence it created, tracked by the variable `iterationCount`.

Using the iterator then looks like this:

> [!NOTE]

> It is not possible to know reflectively whether a particular object is an iterator. If you need to do this, use Iterables.

Contoh Kode / Perintah CLI:

```
function makeRangeIterator(start = 0, end = Infinity, step = 1) {
  let nextIndex = start;
  let iterationCount = 0;

  const rangeIterator = {
    next() {
      let result;
      if (nextIndex < end) {
        result = { value: nextIndex, done: false };
        nextIndex += step;
        iterationCount++;
        return result;
      }
      return { value: iterationCount, done: true };
    },
  };
  return rangeIterator;
}

// ----------------------------------------

const iter = makeRangeIterator(1, 10, 2);

let result = iter.next();
while (!result.done) {
  console.log(result.value); // 1 3 5 7 9
  result = iter.next();
}
```

```
console.log("Iterated over sequence of size:", result.value); // [5 numbers returned, that took interval in between: 0 to 10]
```

[javascript_047] JavaScript - Iterators and Generators

Iterators and Generators (yield, Iterables, for...of loops): Generator functions

Sumber: ht

Generator functions

While custom iterators are a useful tool, their creation requires careful programming due to the need to explicitly maintain their internal state. **Generator functions** provide a powerful alternative: they allow you to define an iterative algorithm by writing a single function whose execution is not continuous. Generator functions are written using the {{jsxref("Statements/function*", "function*")}} syntax.

When called, generator functions do not initially execute their code. Instead, they return a special type of iterator, called a **Generator**. When a value is consumed by calling the generator's `next` method, the Generator function executes until it encounters the `yield` keyword.

The function can be called as many times as desired, and returns a new Generator each time. Each Generator may only be iterated once.

We can now adapt the example from above. The behavior of this code is identical, but the implementation is much easier to write and read.

Contoh Kode / Perintah CLI:

```
function* makeRangeIterator(start = 0, end = Infinity, step = 1) {
  let iterationCount = 0;
  for (let i = start; i < end; i += step) {
    iterationCount++;
    yield i;
  }
  return iterationCount;
}
```

[javascript_048] JavaScript - Iterators and Generators

Iterators and Generators (yield, Iterables, for...of loops): Iterables

Sumber: ht

Iterables

An object is **iterable** if it defines its iteration behavior, such as what values are looped over in a {{jsxref("Statements/for...of", "for...of")}} construct. Some built-in types, such as or , have a default iteration behavior, while other types (such as) do not.

In order to be **iterable**, an object must implement the `[Symbol.iterator]()` method. This means that the object (or one of the objects up its prototype chain) must have a property with a key.

It may be possible to iterate over an iterable more than once, or only once. It is up to the programmer to know which is the case.

Iterables which can iterate only once (such as Generators) customarily return `this` from their `[Symbol.iterator]()` method, whereas iterables which can be iterated many times must return a new iterator on each invocation of `[Symbol.iterator]()`.

User-defined iterables

You can make your own iterables like this:

User-defined iterables can be used in `for...of` loops or the spread syntax as usual.

Built-in iterables

, , , and are all built-in iterables, because their prototype objects all have a method.

Syntaxes expecting iterables

Some statements and expressions expect iterables. For example: the {{jsxref("Statements/for...of", "for...of")}} loops, {{jsxref("Operators/Spread_syntax", "spread syntax", "", 1)}}, {{jsxref("Operators/yield*", "yield*")}}, and {{jsxref("Operators/Destructuring", "destructuring", "", 1)}} syntax.

Contoh Kode / Perintah CLI:

```
function* makeIterator() {
  yield 1;
  yield 2;
}

const iter = makeIterator();

for (const itItem of iter) {
  console.log(itItem);
}

console.log(iter[Symbol.iterator]() === iter); // true

// This example shows us generator(iterator) is an iterable object,
// which has the [Symbol.iterator]() method return the `iter` (itself),
// and consequently, the `iter` object can iterate only _once_.

// If we change the [Symbol.iterator]() method of `iter` to a function/generator
// which returns a new iterator/generator object, `iter`
// can iterate many times

iter[Symbol.iterator] = function* () {
  yield 2;
  yield 1;
};

// ----------------------------------------

const myIterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  },
};

// ----------------------------------------

for (const value of myIterable) {
  console.log(value);
}
// 1
// 2
// 3

[...myIterable]; // [1, 2, 3]
```

[javascript_049] JavaScript - Iterators and Generators

Iterators and Generators (yield, Iterables, for...of loops): Advanced generators

Sumber: ht

Advanced generators

Generators compute their `yield`ed values _on demand_, which allows them to efficiently represent sequences that are expensive to compute (or even infinite sequences, as demonstrated above).

The {{jsxref("Generator/next", "next()")}} method also accepts a value, which can be used to modify the internal state of the generator. A value passed to `next()` will be received by `yield`.

> [!NOTE]

> A value passed to the _first_ invocation of `next()` is always ignored.

Here is the fibonacci generator using `next(x)` to restart the sequence:
You can force a generator to throw an exception by calling its {{jsxref("Generator/throw", "throw()")}} method and passing the exception value it should throw. This exception will be thrown from the current suspended context of the generator, as if the `yield` that is currently suspended were instead a `throw value` statement.
If the exception is not caught from within the generator, it will propagate up through the call to `throw()`, and subsequent calls to `next()` will result in the `done` property being `true`.
Generators have a {{jsxref("Generator/return", "return()")}} method that returns the given value and finishes the generator itself.
{{PreviousNext("Web/JavaScript/Guide/Typed_arrays", "Web/JavaScript/Guide/Resource_management")}}

Contoh Kode / Perintah CLI:

```
function* fibonacci() {
  let current = 0;
  let next = 1;
  while (true) {
    const reset = yield current;
    [current, next] = [next, next + current];
    if (reset) {
      current = 0;
      next = 1;
    }
  }
}

const sequence = fibonacci();
console.log(sequence.next().value); // 0
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 2
console.log(sequence.next().value); // 3
console.log(sequence.next().value); // 5
console.log(sequence.next().value); // 8
console.log(sequence.next(true).value); // 0
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 2
```

[javascript_050] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Overview

Sumber: ht

{{Previous("Web/JavaScript/Guide/Internationalization")}}
This guide gives you all you need to get started with JavaScript module syntax.

[javascript_051] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): A background on modules

Sumber: ht

A background on modules
JavaScript programs started off pretty small -- most of its usage in the early days was to do isolated scripting tasks, providing a bit of interactivity to your web pages where needed, so large scripts were generally not needed. Fast forward a few years and we now have complete applications being run in browsers with a lot of JavaScript, as well as JavaScript being used in other contexts (Node.js, for example).
Complex projects necessitate a mechanism for splitting JavaScript programs into separate modules that can be imported when needed. Node.js has had this ability for a long time, and there are a number of JavaScript libraries and frameworks that enable module usage (for example, other CommonJS and AMD-based module systems like

RequireJS, webpack, and Babel).
All modern browsers support module features natively without needing transpilation. It can only be a good thing -- browsers can optimize loading of modules, making it more efficient than having to use a library and do all of that extra client-side processing and extra round trips. It does not obsolete bundlers like webpack, though -- bundlers still do a good job at partitioning code into reasonably sized chunks, and are able to do other optimizations like minification, dead code elimination, and tree-shaking.

[javascript_052] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Introducing an example

Sumber: ht

Introducing an example
To demonstrate usage of modules, we've created a set of examples that you can find on GitHub. These examples demonstrate a set of modules that create a `` element on a webpage, and then draw (and report information about) different shapes on the canvas.
These are fairly trivial, but have been kept deliberately simple to demonstrate modules clearly.
> [!NOTE]
> If you want to download the examples and run them locally, you'll need to run them through a local web server.

[javascript_053] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Basic example structure

Sumber: ht

Basic example structure
In our first example (see basic-modules) we have a file structure as follows:
> [!NOTE]
> All of the examples in this guide have basically the same structure; the above should start getting pretty familiar.
The modules directory's two modules are described below:
- `canvas.js` -- contains functions related to setting up the canvas:
- `create()` -- creates a canvas with a specified `width` and `height` inside a wrapper `` with a specified ID, which is itself appended inside a specified parent element. Returns an object containing the canvas's 2D context and the wrapper's ID.
- `createReportList()` -- creates an unordered list appended inside a specified wrapper element, which can be used to output report data into. Returns the list's ID.
- `square.js` -- contains:
- `name` -- a constant containing the string 'square'.
- `draw()` -- draws a square on a specified canvas, with a specified size, position, and color. Returns an object containing the square's size, position, and color.
- `reportArea()` -- writes a square's area to a specific report list, given its length.
- `reportPerimeter()` -- writes a square's perimeter to a specific report list, given its length.
Aside -- .mjs versus .js
Throughout this article, we've used `.js` extensions for our module files, but in other resources you may see the `.mjs` extension used instead. V8's documentation recommends this, for example. The reasons given are:
- It is good for clarity, i.e., it makes it clear which files are modules, and which are regular JavaScript.
- It ensures that your module files are parsed as a module by runtimes such as Node.js, and build tools such as Babel.
However, we decided to keep using `.js`, at least for the moment. To get modules to work correctly in a browser, you need to mak

[javascript_054] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Exporting module features

Sumber: ht

Exporting module features

The first thing you do to get access to module features is export them. This is done using the {{jsxref("Statements/export", "export")}} statement.

The easiest way to use it is to place it in front of any items you want exported out of the module, for example:

You can export functions, `var`, `let`, `const`, and -- as we'll see later -- classes. They need to be top-level items: for example, you can't use `export` inside a function.

A more convenient way of exporting all the items you want to export is to use a single export statement at the end of your module file, followed by a comma-separated list of the features you want to export wrapped in curly braces. For example:

Contoh Kode / Perintah CLI:

```
export const name = "square";

export function draw(ctx, length, x, y, color) {
  ctx.fillStyle = color;
  ctx.fillRect(x, y, length, length);

  return { length, x, y, color };
}

// ------------------------------------------

export { name, draw, reportArea, reportPerimeter };
```

[javascript_055] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Importing features into your script

Sumber: ht

Importing features into your script

Once you've exported some features out of your module, you need to import them into your script to be able to use them. The simplest way to do this is as follows:

You use the {{jsxref("Statements/import", "import")}} statement, followed by a comma-separated list of the features you want to import wrapped in curly braces, followed by the keyword `from`, followed by the _module specifier_.

The _module specifier_ provides a string that the JavaScript environment can resolve to a path to the module file.

In a browser, this could be a path relative to the site root, which for our `basic-modules` example would be `/js-examples/module-examples/basic-modules`.

However, here we are instead using the dot (`.`) syntax to mean "the current location", followed by the relative path to the file we are trying to find. This is much better than writing out the entire absolute path each time, as relative paths are shorter and make the URL portable -- the example will still work if you move it to a different location in the site hierarchy.

So for example:

becomes

You can see such lines in action in `main.js`.

> [!NOTE]

> In some module systems, you can use a module specifier like `modules/square` that isn't a relative or absolute path, and that doesn't have a file extension.

> This kind of specifier can be used in a browser environment if you first define an import map.

Once you've imported the features into your script, you can use them just like they were defined inside the same file. The following is found in `main.js`, below the import lines:

> [!NOTE]
> The imported values are read-only views of the features that were exported. Similar to `const` variables, you cannot re-assign the variable that was imported, but you can still modify properties of

Contoh Kode / Perintah CLI:

```
import { name, draw, reportArea, reportPerimeter } from "./modules/square.js";

// ----------------------------------------

becomes

// ----------------------------------------

You can see such lines in action in [`main.js`](https://github.com/mdn/js-examples/blob/main/module-examples/basic-modules/main.js).

> [!NOTE]
> In some module systems, you can use a module specifier like `modules/square` that isn't a relative or absolute path, and that doesn't have a file extension.
> This kind of specifier can be used in a browser environment if you first define an [import map](#importing_modules_using_import_maps).

Once you've imported the features into your script, you can use them just like they were defined inside the same file. The following is found in `main.js`, below the import lines:
```

[javascript_056] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Importing modules using import maps

Sumber: ht

Importing modules using import maps
Above we saw how a browser can import a module using a module specifier that is either an absolute URL, or a relative URL that is resolved using the base URL of the document:
Import maps allow developers to instead specify almost any text they want in the module specifier when importing a module; the map provides a corresponding value that will replace the text when the module URL is resolved.
For example, the `imports` key in the import map below defines a "module specifier map" JSON object where the property names can be used as module specifiers, and the corresponding values will be substituted when the browser resolves the module URL.
The values must be absolute or relative URLs.
Relative URLs are resolved to absolute URL addresses using the base URL of the document containing the import map.
The import map is defined using a JSON object inside a `` element with the `type` attribute set to `importmap`.
Note that an import map only applies to the document -- the specification does not cover how to apply an import map in a worker or worklet context.
With this map you can now use the property names above as module specifiers.
If there is no trailing forward slash on the module specifier key then the whole module specifier key is matched and substituted.
For example, below we match bare module names, and remap a URL to another path.
If the module specifier has a trailing forward slash then the value must have one as well, and the key is matched as a "path prefix".
This allows remapping of whole classes of URLs.
It is possible for multiple keys in an import map to be valid matches for a module specifier.
For example, a module specifier of `shapes/circle/` could match the module specifier keys `shapes/` and `shapes/circle/`.
In this c

Contoh Kode / Perintah CLI:

```
import { name as circleName } from "https://example.com/shapes/circle.js";
```

```
import { name as squareName, draw } from "./shapes/square.js";

// ----------------------------------------

<script type="importmap">
  {
    "imports": {
      "shapes": "./shapes/square.js",
      "shapes/square": "./modules/shapes/square.js",
      "https://example.com/shapes/square.js": "./shapes/square.js",
      "https://example.com/shapes/": "/shapes/square/",
      "../shapes/square": "./shapes/square.js"
    }
  }
</script>

// ----------------------------------------

// Bare module names as module specifiers
import { name as squareNameOne } from "shapes";
import { name as squareNameTwo } from "shapes/square";

// Remap a URL to another URL
import { name as squareNameThree } from "https://example.com/shapes/square.js";
```

[javascript_057] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Loading non-JavaScript resources

Sumber: ht

Loading non-JavaScript resources

One exciting feature that a unified module architecture brings is the ability to load non-JavaScript resources as modules. For example, you can import JSON as a JavaScript object, or import CSS as a object.

You must explicitly declare what kind of resource you are importing. By default, the browser assumes that the resource is JavaScript, and will throw an error if the resolved resource is something else. To import JSON, CSS, or other types of resource, use the import attributes syntax:

Browsers will also perform validation on the module type, and fail if, for example, `./data.json` does not resolve to a JSON file. This ensures that you don't accidentally execute code when you just intend to import data. Once imported successfully, you can now use the imported value as a normal JavaScript object or `CSSStyleSheet` object.

Contoh Kode / Perintah CLI:

```
import colors from "./colors.json" with { type: "json" };
import styles from "./styles.css" with { type: "css" };

// ----------------------------------------

console.log(colors.map((color) => color.value));
document.adoptedStyleSheets = [styles];
```

[javascript_058] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Applying the module to your HTML

Sumber: ht

Applying the module to your HTML

Now we just need to apply the `main.js` module to our HTML page. This is very similar to how we apply a regular script to a page, with a few notable differences.

First of all, you need to include `type="module"` in the `` element, to declare this script as a module. To import the `main.js` script, we use this:
You can also embed the module's script directly into the HTML file by placing the JavaScript code within the body of the `` element:
You can only use `import` and `export` statements inside modules, not regular scripts. An error will be thrown if your `` element doesn't have the `type="module"` attribute and attempts to import other modules. For example:
You should generally define all your modules in separate files. Modules declared inline in HTML can only import other modules, but anything they export will not be accessible by other modules (because they don't have a URL).
> [!NOTE]
> Modules and their dependencies can be preloaded by specifying them in `` elements with `rel="modulepreload"`.
> This can significantly reduce load time when the modules are used.

Contoh Kode / Perintah CLI:

```
<script type="module" src="main.js"></script>

// ----------------------------------------

<script type="module">
  /* JavaScript module code here */
</script>
```

[javascript_059] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Other differences between modules and classic scripts

Sumber: ht

Other differences between modules and classic scripts
- You need to pay attention to local testing -- if you try to load the HTML file locally (i.e., with a `file://` URL), you'll run into CORS errors due to JavaScript module security requirements. You need to do your testing through a server.
- Also, note that you might get different behavior from sections of script defined inside modules as opposed to in classic scripts. This is because modules use automatically.
- There is no need to use the `defer` attribute (see `` attributes) when loading a module script; modules are deferred automatically.
- Modules are only executed once, even if they have been referenced in multiple `` tags.
- Last but not least, let's make this clear -- module features are imported into the scope of a single script -- they aren't available in the global scope. Therefore, you will only be able to access imported features in the script they are imported into, and you won't be able to access them from the JavaScript console, for example. You'll still get syntax errors shown in the DevTools, but you'll not be able to use some of the debugging techniques you might have expected to use.
Module-defined variables are scoped to the module unless explicitly attached to the global object. On the other hand, globally-defined variables are available within the module. For example, given the following code:
The page would still render `Hello`, because the global variables `text` and `document` are available in the module. (Also note from this example that a module doesn't necessarily need an import/export statement -- the only thing needed is for the entry point to have `type="module"`.)

Contoh Kode / Perintah CLI:

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <title>Example page</title>
    <link rel="stylesheet" href="" />
  </head>
  <body>
    <div id="main"></div>
    <script>
      // A var statement creates a global variable.
```

```
      var text = "Hello";
    </script>
    <script type="module" src="./render.js"></script>
  </body>
</html>

// ----------------------------------------

/* render.js */
document.getElementById("main").innerText = text;
```

[javascript_060] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Default exports versus named exports

Sumber: ht

Default exports versus named exports
The functionality we've exported so far has been comprised of **named exports** -- each item (be it a function, `const`, etc.) has been referred to by its name upon export, and that name has been used to refer to it on import as well.
There is also a type of export called the **default export** -- this is designed to make it easy to have a default function provided by a module, and also helps JavaScript modules to interoperate with existing CommonJS and AMD module systems (as explained nicely in ES6 In Depth: Modules by Jason Orendorff; search for "Default exports").
Let's look at an example as we explain how it works. In our basic-modules `square.js` you can find a function called `randomSquare()` that creates a square with a random color, size, and position. We want to export this as our default, so at the bottom of the file we write this:
Note the lack of curly braces.
We could instead prepend `export default` onto the function and define it as an anonymous function, like this:
Over in our `main.js` file, we import the default function using this line:
Again, note the lack of curly braces. This is because there is only one default export allowed per module, and we know that `randomSquare` is it. The above line is basically shorthand for:
> [!NOTE]
> The as syntax for renaming exported items is explained below in the Renaming imports and exports section.

Contoh Kode / Perintah CLI:

```
export default randomSquare;

// ----------------------------------------

export default function (ctx) {
  // ...
}

// ----------------------------------------

import randomSquare from "./modules/square.js";
```

[javascript_061] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Avoiding naming conflicts

Sumber: ht

Avoiding naming conflicts
So far, our canvas shape drawing modules seem to be working OK. But what happens if we try to add a module that deals with drawing another shape, like a circle or triangle? These shapes would probably have associated functions like `draw()`, `reportArea()`, etc. too; if we tried to import different functions of the same name into the same top-level module file, we'd end up with conflicts and errors.

Fortunately there are a number of ways to get around this. We'll look at these in the following sections.

[javascript_062] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Renaming imports and exports

Sumber: ht

Renaming imports and exports

Inside your `import` and `export` statement's curly braces, you can use the keyword `as` along with a new feature name, to change the identifying name you will use for a feature inside the top-level module.

So for example, both of the following would do the same job, albeit in a slightly different way:

Let's look at a real example. In our renaming directory you'll see the same module system as in the previous example, except that we've added `circle.js` and `triangle.js` modules to draw and report on circles and triangles.

Inside each of these modules, we've got features with the same names being exported, and therefore each has the same `export` statement at the bottom:

When importing these into `main.js`, if we tried to use

The browser would throw an error such as "SyntaxError: redeclaration of import name" (Firefox).

Instead we need to rename the imports so that they are unique:

Note that you could solve the problem in the module files instead, e.g.

And it would work just the same. What style you use is up to you, however it arguably makes more sense to leave your module code alone, and make the changes in the imports. This especially makes sense when you are importing from third party modules that you don't have any control over.

Contoh Kode / Perintah CLI:

```
// -- module.js --
export { function1 as newFunctionName, function2 as anotherNewFunctionName };

// -- main.js --
import { newFunctionName, anotherNewFunctionName } from "./modules/module.js";

// ----------------------------------------

// -- module.js --
export { function1, function2 };

// -- main.js --
import {
  function1 as newFunctionName,
  function2 as anotherNewFunctionName,
} from "./modules/module.js";

// ----------------------------------------

export { name, draw, reportArea, reportPerimeter };
```

[javascript_063] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Creating a module object

Sumber: ht

Creating a module object

The above method works OK, but it's a little messy and long-winded. An even better solution is to import each module's features inside a module object. The following syntax form does that:

This grabs all the exports available inside `module.js`, and makes them available as members of an object `Module`, effectively giving it its own namespace. So for example:

Again, let's look at a real example. If you go to our module-objects directory, you'll see the same example again, but

rewritten to take advantage of this new syntax. In the modules, the exports are all in the following simple form:
The imports on the other hand look like this:
In each case, you can now access the module's imports underneath the specified object name, for example:
So you can now write the code just the same as before (as long as you include the object names where needed), and the imports are much neater.

Contoh Kode / Perintah CLI:

```
import * as Module from "./modules/module.js";

// ----------------------------------------

Module.function1();
Module.function2();

// ----------------------------------------

export { name, draw, reportArea, reportPerimeter };
```

[javascript_064] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Modules and classes

Sumber: ht

Modules and classes
As we hinted at earlier, you can also export and import classes; this is another option for avoiding conflicts in your code, and is especially useful if you've already got your module code written in an object-oriented style.
You can see an example of our shape drawing module rewritten with ES classes in our classes directory. As an example, the `square.js` file now contains all its functionality in a single class:
which we then export:
Over in `main.js`, we import it like this:
And then use the class to draw our square:

Contoh Kode / Perintah CLI:

```
class Square {
  constructor(ctx, listId, length, x, y, color) {
    // ...
  }

  draw() {
    // ...
  }

  // ...
}

// ----------------------------------------

export { Square };

// ----------------------------------------

import { Square } from "./modules/square.js";
```

[javascript_065] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Aggregating modules

Sumber: ht

Aggregating modules

There will be times where you'll want to aggregate modules together. You might have multiple levels of dependencies, where you want to simplify things, combining several submodules into one parent module. This is possible using export syntax of the following forms in the parent module:

For an example, see our module-aggregation directory. In this example (based on our earlier classes example) we've got an extra module called `shapes.js`, which aggregates all the functionality from `circle.js`, `square.js`, and `triangle.js` together. We've also moved our submodules inside a subdirectory inside the `modules` directory called `shapes`. So the module structure in this example is:

In each of the submodules, the export is of the same form, e.g.

Next up comes the aggregation part. Inside `shapes.js`, we include the following lines:

These grab the exports from the individual submodules and effectively make them available from the `shapes.js` module.

> [!NOTE]

> The exports referenced in `shapes.js` basically get redirected through the file and don't really exist there, so you won't be able to write any useful related code inside the same file.

So now in the `main.js` file, we can get access to all three module classes by replacing

with the following single line:

Contoh Kode / Perintah CLI:

```
export * from "x.js";
export { name } from "x.js";

// ----------------------------------------

In each of the submodules, the export is of the same form, e.g.

// ----------------------------------------

Next up comes the aggregation part. Inside [`shapes.js`](https://github.com/mdn/js-examples/blob/main/module-examples/module-aggregation/modules/shapes.js), we include the following lines:
```

[javascript_066] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Dynamic module loading

Sumber: ht

Dynamic module loading

A recent addition to JavaScript modules functionality is dynamic module loading. This allows you to dynamically load modules only when they are needed, rather than having to load everything up front. This has some obvious performance advantages; let's read on and see how it works.

This new functionality allows you to call `import()` as a function, passing it the path to the module as a parameter. It returns a , which fulfills with a module object (see Creating a module object) giving you access to that object's exports. For example:

> [!NOTE]

> Dynamic import is permitted in the browser main thread, and in shared and dedicated workers.

> However `import()` will throw if called in a service worker or worklet.

Let's look at an example. In the dynamic-module-imports directory we've got another example based on our classes example. This time however we are not drawing anything on the canvas when the example loads. Instead, we include three buttons -- "Circle", "Square", and "Triangle" -- that, when pressed, dynamically load the required module and

then use it to draw the associated shape.

In this example we've only made changes to our `index.html` and `main.js` files -- the module exports remain the same as before.

Over in `main.js` we've grabbed a reference to each button using a `document.querySelector()` call, for example:

We then attach an event listener to each button so that when pressed, the relevant module is dynamically loaded and used to draw the shape:

Note that, because the promise fulfillment returns a module object, the class is then made a subfeature of the object, hence we now need to access the constructor with `Module.` prepended to it, e.g., `Module.Square( /* ... */ )`.

Another advantage of dynamic imports is that they are always ava

Contoh Kode / Perintah CLI:

```
import("./modules/myModule.js").then((module) => {
  // Do something with the module.
});

// ----------------------------------------

const squareBtn = document.querySelector(".square");

// ----------------------------------------

squareBtn.addEventListener("click", () => {
  import("./modules/square.js").then((Module) => {
    const square = new Module.Square(
      myCanvas.ctx,
      myCanvas.listId,
      50,
      50,
      100,
      "blue",
    );
    square.draw();
    square.reportArea();
    square.reportPerimeter();
  });
});
```

[javascript_067] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Top level await

Sumber: ht

Top level await

Top level await is a feature available within modules. This means the `await` keyword can be used. It allows modules to act as big asynchronous functions meaning code can be evaluated before use in parent modules, but without blocking sibling modules from loading.

Let's take a look at an example. You can find all the files and code described in this section within the `top-level-await` directory, which extends from the previous examples.

Firstly we'll declare our color palette in a separate `colors.json` file:

Then we'll create a module called `getColors.js` which uses a fetch request to load the `colors.json` file and return the data as an object.

Notice the last export line here.

We're using the keyword `await` before specifying the constant `colors` to export. This means any other modules which include this one will wait until `colors` has been downloaded and parsed before using it.

Let's include this module in our `main.js` file:

We'll use `colors` instead of the previously used strings when calling our shape functions:

This is useful because the code within `main.js` won't execute until the code in `getColors.js` has run. However it won't block other modules being loaded. For instance our `canvas.js` module will continue to load while `colors` is being

fetched.

Contoh Kode / Perintah CLI:

```
Then we'll create a module called [`getColors.js`](https://github.com/mdn/js-examples/blob/main/module-examples/top-level-await/modules/getColors.js) which uses a fetch request to load the [`colors.json`](https://github.com/mdn/js-examples/blob/main/module-examples/top-level-await/data/colors.json) file and return the data as an object.

// ----------------------------------------

Notice the last export line here.

We're using the keyword `await` before specifying the constant `colors` to export. This means any other modules which include this one will wait until `colors` has been downloaded and parsed before using it.

Let's include this module in our [`main.js`](https://github.com/mdn/js-examples/blob/main/module-examples/top-level-await/main.js) file:

// ----------------------------------------

We'll use `colors` instead of the previously used strings when calling our shape functions:
```

[javascript_068] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Import declarations are hoisted

Sumber: ht

Import declarations are hoisted

Import declarations are hoisted. In this case, it means that the imported values are available in the module's code even before the place that declares them, and that the imported module's side effects are produced before the rest of the module's code starts running.

So for example, in `main.js`, importing `Canvas` in the middle of the code would still work:

Still, it is considered good practice to put all your imports at the top of the code, which makes it easier to analyze dependencies.

Contoh Kode / Perintah CLI:

```
// ...
const myCanvas = new Canvas("myCanvas", document.body, 480, 320);
myCanvas.create();
import { Canvas } from "./modules/canvas.js";
myCanvas.createReportList();
// ...
```

[javascript_069] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Cyclic imports

Sumber: ht

Cyclic imports

Modules can import other modules, and those modules can import other modules, and so on. This forms a directed graph called the "dependency graph". In an ideal world, this graph is acyclic. In this case, the graph can be evaluated using a depth-first traversal.

However, cycles are often inevitable. Cyclic import arises if module `a` imports module `b`, but `b` directly or indirectly depends on `a`. For example:

Cyclic imports don't always fail. The imported variable's value is only retrieved when the variable is actually used (hence allowing live bindings), and only if the variable remains uninitialized at that time will a `ReferenceError` be thrown.

In this example, both `a` and `b` are used asynchronously. Therefore, at the time the module is evaluated, neither `b` nor `a` is actually read, so the rest of the code is executed as normal, and the two `export` declarations produce the

values of `a` and `b`. Then, after the timeout, both `a` and `b` are available, so the two `console.log` statements also execute as normal.

If you change the code to use `a` synchronously, the module evaluation fails:

This is because when JavaScript evaluates `a.js`, it needs to first evaluate `b.js`, the dependency of `a.js`. However, `b.js` uses `a`, which is not yet available.

On the other hand, if you change the code to use `b` synchronously but `a` asynchronously, the module evaluation succeeds:

This is because the evaluation of `b.js` completes normally, so the value of `b` is available when `a.js` is evaluated.

You should usually avoid cyclic imports in your project, because they make your code more error-prone. Some common cycle-elimination techniques are:

- Merge the two modules into one.
- Move the shared code into a third module.
- Move some code from one module

Contoh Kode / Perintah CLI:

```
// -- a.js --
import { b } from "./b.js";

// -- b.js --
import { a } from "./a.js";

// Cycle:
// a.js ???> b.js
//  ^         ?
//  ??????????

// ----------------------------------------

// -- a.js --
import { b } from "./b.js";

setTimeout(() => {
  console.log(b); // 1
}, 10);

export const a = 2;

// -- b.js --
import { a } from "./a.js";

setTimeout(() => {
  console.log(a); // 2
}, 10);

export const b = 1;

// ----------------------------------------

// -- a.js (entry module) --
import { b } from "./b.js";

export const a = 2;

// -- b.js --
import { a } from "./a.js";

console.log(a); // ReferenceError: Cannot access 'a' before initialization
export const b = 1;
```

[javascript_070] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Authoring "isomorphic" modules

Sumber: ht

Authoring "isomorphic" modules

The introduction of modules encourages the JavaScript ecosystem to distribute and reuse code in a modular fashion. However, that doesn't necessarily mean a piece of JavaScript code can run in every environment. Suppose you discovered a module that generates SHA hashes of your user's password. Can you use it in the browser front end? Can you use it on your Node.js server? The answer is: it depends.

Modules still have access to global variables, as demonstrated previously. If the module references globals like `window`, it can run in the browser, but will throw an error in your Node.js server, because `window` is not available there. Similarly, if the code requires access to `process` to be functional, it can only be used in Node.js.

In order to maximize the reusability of a module, it is often advised to make the code "isomorphic" -- that is, exhibits the same behavior in every runtime. This is commonly achieved in three ways:

- Separate your modules into "core" and "binding". For the "core", focus on pure JavaScript logic like computing the hash, without any DOM, network, filesystem access, and expose utility functions. For the "binding" part, you can read from and write to the global context. For example, the "browser binding" may choose to read the value from an input box, while the "Node binding" may read it from `process.env`, but values read from either place will be piped to the same core function and handled in the same way. The core can be imported in every environment and used in the same way, while only the binding, which is usually lightweight, needs to be platform-specific.
- Detect whether a particular global exists before using it. For example, if you test that `typeof window === "undefined"`, you know that you are probably

Contoh Kode / Perintah CLI:

```
// myModule.js
  let password;
  if (typeof process !== "undefined") {
    // We are running in Node.js; read it from `process.env`
    password = process.env.PASSWORD;
  } else if (typeof window !== "undefined") {
    // We are running in the browser; read it from the input box
    password = document.getElementById("password").value;
  }

// ----------------------------------------

// myModule.js
  if (typeof fetch === "undefined") {
    // We are running in Node.js; use node-fetch
    globalThis.fetch = (await import("node-fetch")).default;
  }
  // ...
```

[javascript_071] JavaScript - JavaScript Modules (ESM)

JavaScript Modules (import, export, dynamic import, Top-level await): Troubleshooting

Sumber: ht

Troubleshooting

Here are a few tips that may help you if you are having trouble getting your modules to work. Feel free to add to the list if you discover more!

- We mentioned this before, but to reiterate: `.mjs` files need to be loaded with a MIME-type of `text/javascript` (or another JavaScript-compatible MIME-type, but `text/javascript` is recommended), otherwise you'll get a strict MIME type checking error like "The server responded with a non-JavaScript MIME type".
- If you try to load the HTML file locally (i.e., with a `file://` URL), you'll run into CORS errors due to JavaScript module security requirements. You need to do your testing through a server. GitHub pages is ideal as it also serves `.mjs` files

with the correct MIME type.
- Because `.mjs` is a non-standard file extension, some operating systems might not recognize it, or try to replace it with something else. For example, we found that macOS was silently adding on `.js` to the end of `.mjs` files and then automatically hiding the file extension. So all of our files were actually coming out as `x.mjs.js`. Once we turned off automatically hiding file extensions, and trained it to accept `.mjs`, it was OK.

[javascript_072] JavaScript - Modern Features - ES2023/ES2024: Object.groupBy

Object.groupBy() (Grouping array elements into object keys): Overview

Sumber: ht

> [!NOTE]
> In some versions of some browsers, this method was implemented as the method `Array.prototype.group()`. Due to web compatibility issues, it is now implemented as a static method. Check the browser compatibility table for details.
The **`Object.groupBy()`** static method groups the elements of a given iterable according to the string values returned by a provided callback function. The returned object has separate properties for each group, containing arrays with the elements in the group.
This method should be used when group names can be represented by strings. If you need to group elements using a key that is some arbitrary value, use instead.
{{InteractiveExample("JavaScript Demo: Object.groupBy()", "taller")}}

[javascript_073] JavaScript - Modern Features - ES2023/ES2024: Object.groupBy

Object.groupBy() (Grouping array elements into object keys): Syntax

Sumber: ht

Syntax
Parameters
- `items`
- : An iterable (such as an) whose elements will be grouped.
- `callbackFn`
- : A function to execute for each element in the iterable. It should return a value that can get coerced into a property key (string or symbol) indicating the group of the current element. The function is called with the following arguments:
- `element`
- : The current element being processed.
- `index`
- : The index of the current element being processed.
Return value
A `null`-prototype object with properties for all groups, each assigned to an array containing the elements of the associated group.

[javascript_074] JavaScript - Modern Features - ES2023/ES2024: Object.groupBy

Object.groupBy() (Grouping array elements into object keys): Description

Sumber: ht

Description
`Object.groupBy()` calls a provided `callbackFn` function once for each element in an iterable. The callback function should return a string or symbol (values that are neither type are coerced to strings) indicating the group of the associated element. The values returned by `callbackFn` are used as keys for the object returned by `Object.groupBy()`. Each key has an associated array containing all the elements for which the callback returned the same value.
The elements in the returned object and the original iterable are the same (not). Changing the internal structure of the elements will be reflected in both the original iterable and the returned object.

[javascript_075] JavaScript - Modern Features - ES2023/ES2024: Object.groupBy

Object.groupBy() (Grouping array elements into object keys): Examples

Sumber: ht

Examples

Using Object.groupBy()

First we define an array containing objects representing an inventory of different foodstuffs. Each food has a `type` and a `quantity`.

The code below groups the elements by the value of their `type` property.

The arrow function just returns the `type` of each array element each time it is called. Note that the function argument `{ type }` is a basic example of object destructuring syntax for function arguments. This unpacks the `type` property of an object passed as a parameter, and assigns it to a variable named `type` in the body of the function.

This is a very succinct way to access the relevant values of elements within a function.

We can also create groups inferred from values in one or more properties of the elements. Below is a very similar example that puts the items into `ok` or `restock` groups based on the value of the `quantity` field.

Contoh Kode / Perintah CLI:

```
const inventory = [
  { name: "asparagus", type: "vegetables", quantity: 5 },
  { name: "bananas", type: "fruit", quantity: 0 },
  { name: "goat", type: "meat", quantity: 23 },
  { name: "cherries", type: "fruit", quantity: 5 },
  { name: "fish", type: "meat", quantity: 22 },
];

// ----------------------------------------

const result = Object.groupBy(inventory, ({ type }) => type);

/* Result is:
{
  vegetables: [
    { name: 'asparagus', type: 'vegetables', quantity: 5 },
  ],
  fruit: [
    { name: "bananas", type: "fruit", quantity: 0 },
    { name: "cherries", type: "fruit", quantity: 5 }
  ],
  meat: [
    { name: "goat", type: "meat", quantity: 23 },
    { name: "fish", type: "meat", quantity: 22 }
  ]
}
*/

// ----------------------------------------

function myCallback({ quantity }) {
  return quantity > 5 ? "ok" : "restock";
}

const result2 = Object.groupBy(inventory, myCallback);

/* Result is:
{
  restock: [
    { name: "asparagus", type: "vegetables", quantity: 5 },
    { name: "bananas", type: "fruit", quantity: 0 },
    { name: "cherries", type: "fruit", quantity: 5 }
  ],
```

```
  ok: [
    { name: "goat", type: "meat", quantity: 23 },
    { name: "fish", type: "meat", quantity: 22 }
  ]
}
*/
```

[javascript_076] JavaScript - Modern Features - ES2024: Promise.withResolvers

Promise.withResolvers() (Modern Promise resolution without manual callback wiring): Overview

Sumber: ht

The **`Promise.withResolvers()`** static method returns an object containing a new object and two functions to resolve or reject it, corresponding to the two parameters passed to the executor of the {{jsxref("Promise/Promise", "Promise()")}} constructor.

[javascript_077] JavaScript - Modern Features - ES2024: Promise.withResolvers

Promise.withResolvers() (Modern Promise resolution without manual callback wiring): Syntax

Sumber: ht

Syntax
Parameters
None.
Return value
A plain object containing the following properties:
- `promise`
- : A object.
- `resolve`
- : A function that resolves the promise. For its semantics, see the {{jsxref("Promise/Promise", "Promise()")}} constructor reference.
- `reject`
- : A function that rejects the promise. For its semantics, see the {{jsxref("Promise/Promise", "Promise()")}} constructor reference.

[javascript_078] JavaScript - Modern Features - ES2024: Promise.withResolvers

Promise.withResolvers() (Modern Promise resolution without manual callback wiring): Description

Sumber: ht

Description
`Promise.withResolvers()` is exactly equivalent to the following code:
Except that it is more concise and does not require the use of {{jsxref("Statements/let", "let")}}.
The key difference when using `Promise.withResolvers()` is that the resolution and rejection functions now live in the same scope as the promise itself, instead of being created and used once within the executor. This may enable some more advanced use cases, such as when reusing them for recurring events, particularly with streams and queues. This also generally results in less nesting than wrapping a lot of logic within the executor.
`Promise.withResolvers()` is generic and supports subclassing, which means it can be called on subclasses of `Promise`, and the result will contain a promise of the subclass type. To do so, the subclass's constructor must implement the same signature as the `Promise()` constructor -- accepting a single `executor` function that can be called with the `resolve` and `reject` callbacks as parameters.

Contoh Kode / Perintah CLI:

```
let resolve, reject;
```

```
const promise = new Promise((res, rej) => {
  resolve = res;
  reject = rej;
});
```

[javascript_079] JavaScript - Modern Features - ES2024: Promise.withResolvers

Promise.withResolvers() (Modern Promise resolution without manual callback wiring): Examples

Sumber: ht

Examples

Transforming a stream to an async iterable

The use case of `Promise.withResolvers()` is when you have a promise that should be resolved or rejected by some event listener that cannot be wrapped inside the promise executor. The following example transforms a Node.js readable stream to an async iterable. Each `promise` here represents a single batch of data available, and each time the current batch is read, a new promise is created for the next batch. Note how the event listeners are only attached once, but actually call a different version of the `resolve` and `reject` functions each time.

Calling withResolvers() on a non-Promise constructor

`Promise.withResolvers()` is a generic method. It can be called on any constructor that implements the same signature as the `Promise()` constructor. For example, we can call it on a constructor that passes `console.log` as the `resolve` and `reject` functions to `executor`:

Contoh Kode / Perintah CLI:

```
async function* readableToAsyncIterable(stream) {
  let { promise, resolve, reject } = Promise.withResolvers();
  stream.on("error", (error) => reject(error));
  stream.on("end", () => resolve());
  stream.on("readable", () => resolve());

  while (stream.readable) {
    await promise;
    let chunk;
    while ((chunk = stream.read())) {
      yield chunk;
    }
    ({ promise, resolve, reject } = Promise.withResolvers());
  }
}

// ----------------------------------------

class NotPromise {
  constructor(executor) {
    // The "resolve" and "reject" functions behave nothing like the native
    // promise's, but Promise.withResolvers() just returns them, as is.
    executor(
      (value) => console.log("Resolved", value),
      (reason) => console.log("Rejected", reason),
    );
  }

  static withResolvers = Promise.withResolvers;
}

const { promise, resolve, reject } = NotPromise.withResolvers();
resolve("hello");
// Logs: Resolved hello
```

[javascript_080] JavaScript - Modern Features - ES2023: Non-mutating Array Methods

Array.prototype.toSorted() & Change Array by Copy (toReversed, toSpliced, with): Overview

Sumber: ht

The **`toSorted()`** method of instances is the copying version of the {{jsxref("Array/sort", "sort()")}} method. It returns a new array with the elements sorted in ascending order.

[javascript_081] JavaScript - Modern Features - ES2023: Non-mutating Array Methods

Array.prototype.toSorted() & Change Array by Copy (toReversed, toSpliced, with): Syntax

Sumber: ht

Syntax
Parameters
- `compareFn`
- : A function that determines the order of the elements. If omitted, the array elements are converted to strings, then sorted according to each character's Unicode code point value. See {{jsxref("Array/sort", "sort()")}} for more information.
Return value
A new array with the elements sorted in ascending order.

[javascript_082] JavaScript - Modern Features - ES2023: Non-mutating Array Methods

Array.prototype.toSorted() & Change Array by Copy (toReversed, toSpliced, with): Description

Sumber: ht

Description
See {{jsxref("Array/sort", "sort()")}} for more information on the `compareFn` parameter.
When used on sparse arrays, the `toSorted()` method iterates empty slots as if they have the value `undefined`.
The `toSorted()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

[javascript_083] JavaScript - Modern Features - ES2023: Non-mutating Array Methods

Array.prototype.toSorted() & Change Array by Copy (toReversed, toSpliced, with): Examples

Sumber: ht

Examples
Sorting an array
For more usage examples, see {{jsxref("Array/sort", "sort()")}}.
Using toSorted() on sparse arrays
Empty slots are sorted as if they have the value `undefined`. They are always sorted to the end of the array and `compareFn` is not called for them.
Calling toSorted() on non-array objects
The `toSorted()` method reads the `length` property of `this`. It then collects all existing integer-keyed properties in the range of `0` to `length - 1`, sorts them, and writes them into a new array.

Contoh Kode / Perintah CLI:

```
const months = ["Mar", "Jan", "Feb", "Dec"];
const sortedMonths = months.toSorted();
console.log(sortedMonths); // ['Dec', 'Feb', 'Jan', 'Mar']
console.log(months); // ['Mar', 'Jan', 'Feb', 'Dec']

const values = [1, 10, 21, 2];
const sortedValues = values.toSorted((a, b) => a - b);
```

```
console.log(sortedValues); // [1, 2, 10, 21]
console.log(values); // [1, 10, 21, 2]

// ----------------------------------------

console.log(["a", "c", , "b"].toSorted()); // ['a', 'b', 'c', undefined]
console.log([, undefined, "a", "b"].toSorted()); // ["a", "b", undefined, undefined]

// ----------------------------------------

const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
  3: 3, // ignored by toSorted() since length is 3
};
console.log(Array.prototype.toSorted.call(arrayLike));
// [4, 5, undefined]
```